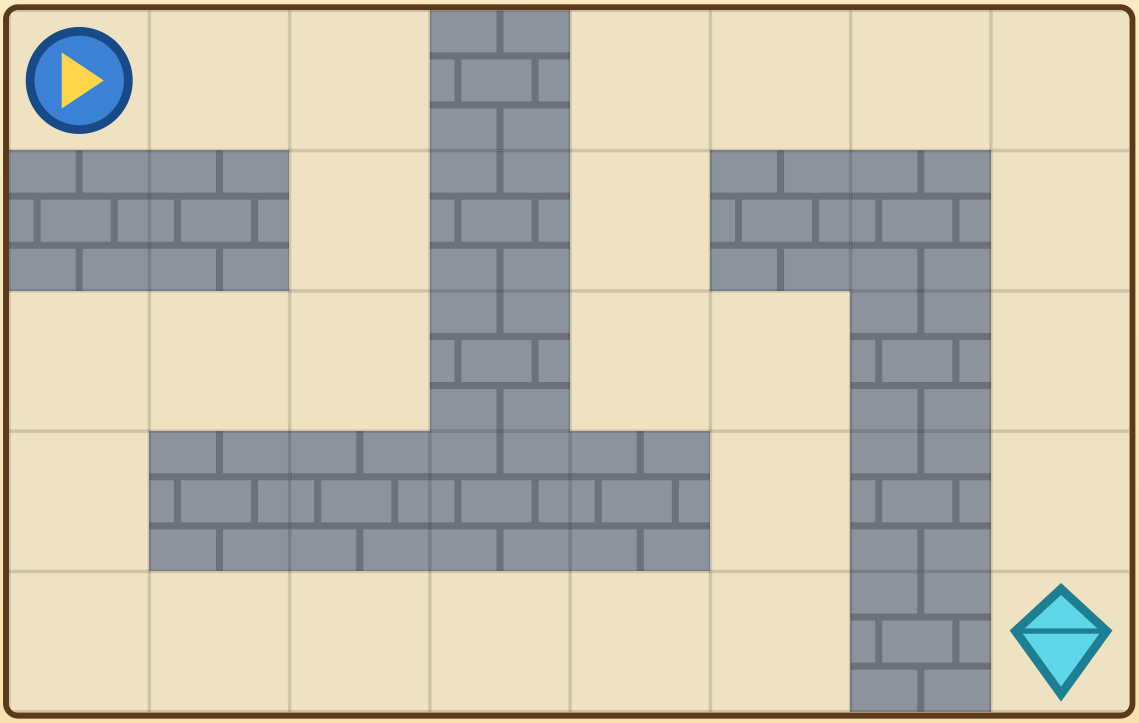
For kids and teachers

Learn Programming with Mazes

From blocks to Python — all 41 maze stages



National Institute of Technology, Maizuru College — Yasuhito Inoue

Contents

How to use this book 2

Part 1 The maze book for kids 3

Chapter 1 What is a program? 4

Chapter 2 The Arrow Maze 5

Chapter 3 The Block Maze 19

Chapter 4 The Keys & Doors Maze 37

Chapter 5 The Dodge-the-Slime Maze 47

Chapter 6 Mazes in Python 57

- Python Arrow Maze 68
- Python Keys & Doors 81
- Python Slime Maze 90

Well done! 100

Part 2 Guide for teachers and parents 101

- About this guide 102
- Overview of the stages 103
- Suggested order and lesson count 105
- Teaching points for each app 106
- Assessment (example) 108
- Using the apps and related materials 109

How to use this book

Use this book together with the “Block Maze” apps on a computer or tablet. You will think about and build programs that move the explorer to the diamond 💎.

1. Read the **problem** and look at the **maze**.
2. Pretend you are the explorer. Trace the path with your finger.
3. Build the blocks in the app and press “▶ Run”. If it doesn’t work, see where it stopped and fix it.
4. Stuck? Look at the 💡 **Hint**. When you are done, turn the page and check the 📖 **Answer** and the ✨ **Key point**.

There is more than one right answer. If you reach the goal, your program is correct, even if it is different from the book. Mistakes are hints — try again and again!

Mazes in this book

Maze	Stages	Grade
Arrow Maze	6	Grade 1+
Block Maze	8	Grade 2+
Keys & Doors Maze	4	Grade 4+
Dodge the Slime Maze	4	Grade 4+
Python mazes (4 apps)	19	Grade 5 – junior high

PART 1

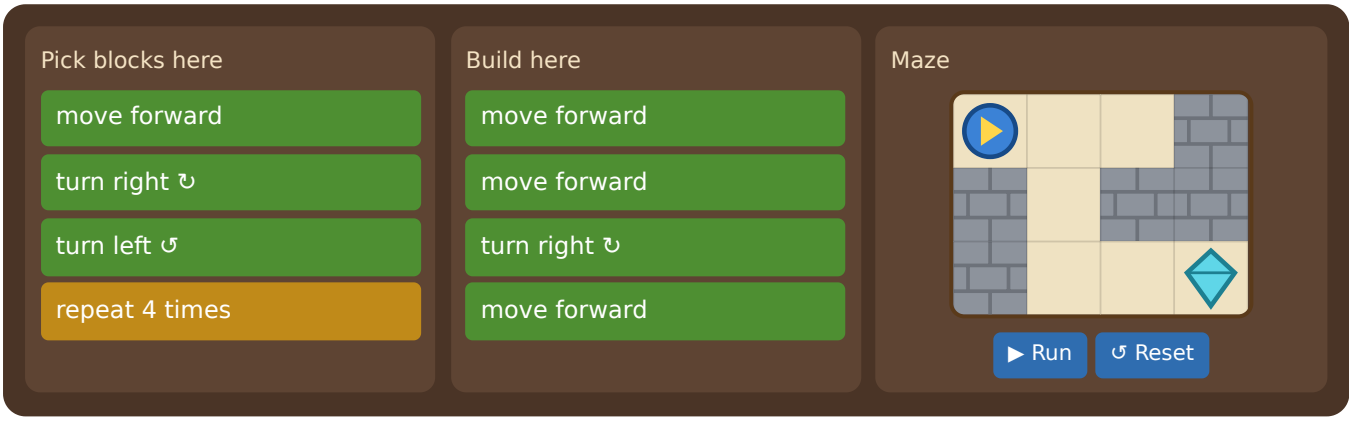
Part 1 The maze book for kids

Chapter 1

What is a program?

A **program** is a “to-do list” for a computer. A computer does exactly what is written, **in order from top to bottom**. It does not do anything that is not written.

In this book you give commands to the explorer and lead them to the goal. You make the commands by snapping “blocks” together.



What the screen looks like (sketch). Drag blocks from the left and stack them.

Which way is “forward”?

“Forward” is the way the explorer is facing. In this book, the triangle ▶ on the explorer points “forward”. “Turn right” means the explorer’s right. Turning the book, or turning your own body, helps.

When it doesn’t work

- Look where the explorer stopped, or where it bumped into a rock.
- Follow the program one block at a time with your finger.
- Finding and fixing mistakes is called **debugging**. Programmers do it every day.

Chapter 2

🧭 The Arrow Maze

Move the explorer with up ⬆, down ⬇, left ⬅ and right ➡ arrows. You don’t need to think about which way the explorer faces. One arrow = one square. This first maze is about **order**.

Commands in this chapter

up ⬆ / down ⬇ / left ⬅ / right ➡ / repeat ○ times (stage 6)

Maze symbols


Start (the explorer). The triangle shows "forward"


Rock (you cannot pass)


Key


Slime (goes back and forth)


Bushes (Arrow Maze)


The goal diamond

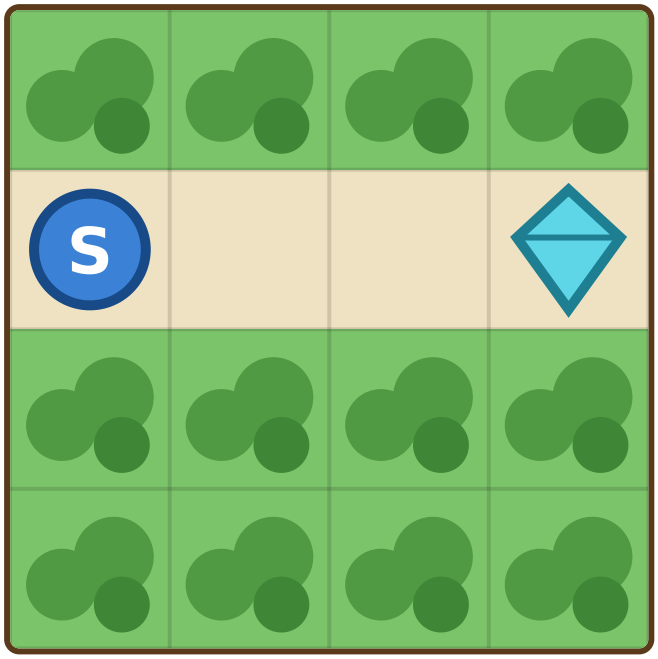

Path


Door (opens with a key)


The slime's track

Stage 1: Go right

Sequence



Problem

Stack three "right ➡" arrows and go to the diamond 💎.

🎯 **Goal** : Line up arrows to move

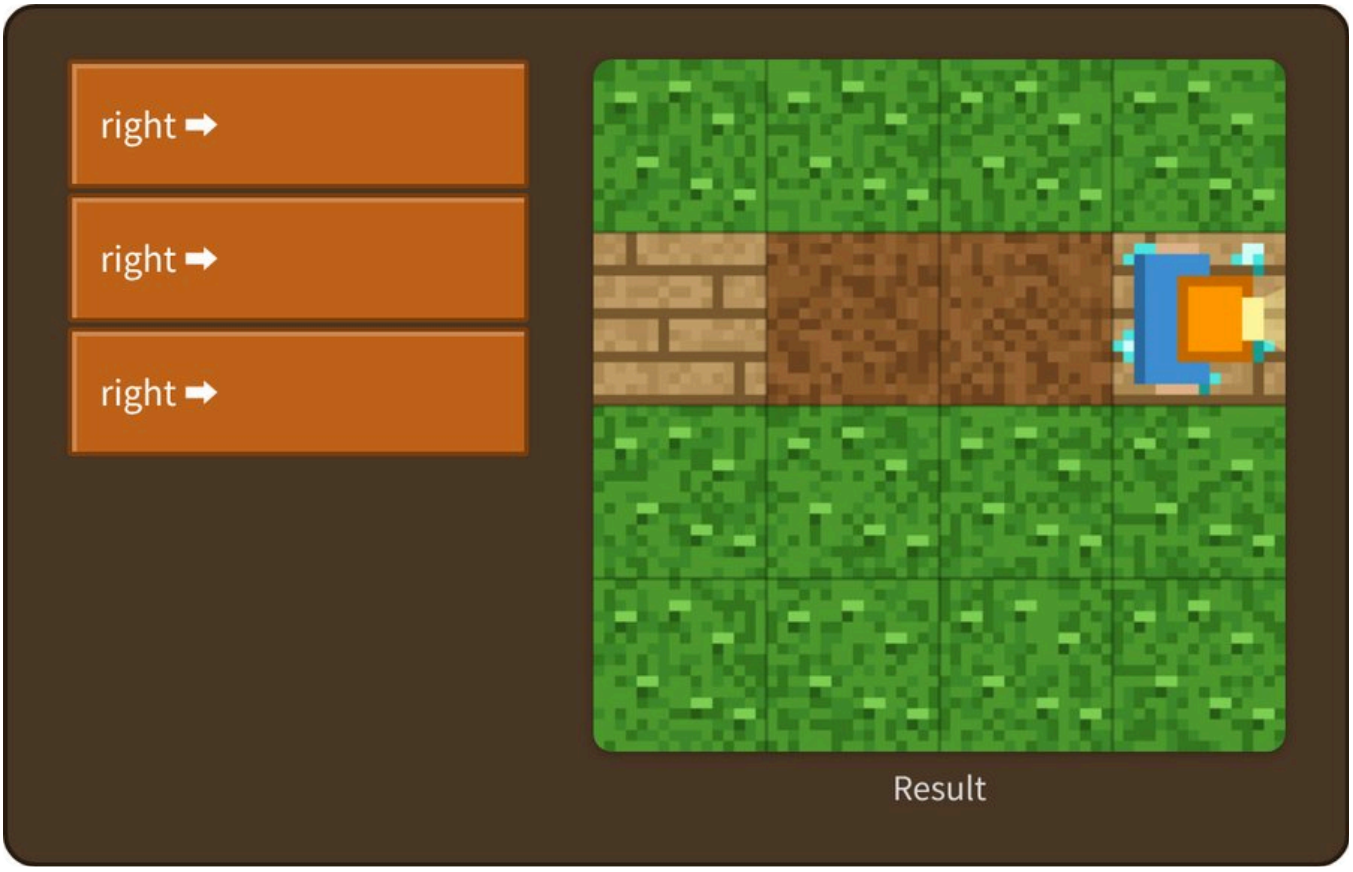
💡 **Hint (look if you get stuck)**

The diamond is 3 squares away.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go right



✨ Key point

One arrow moves one square. Count the squares to the diamond and stack that many.

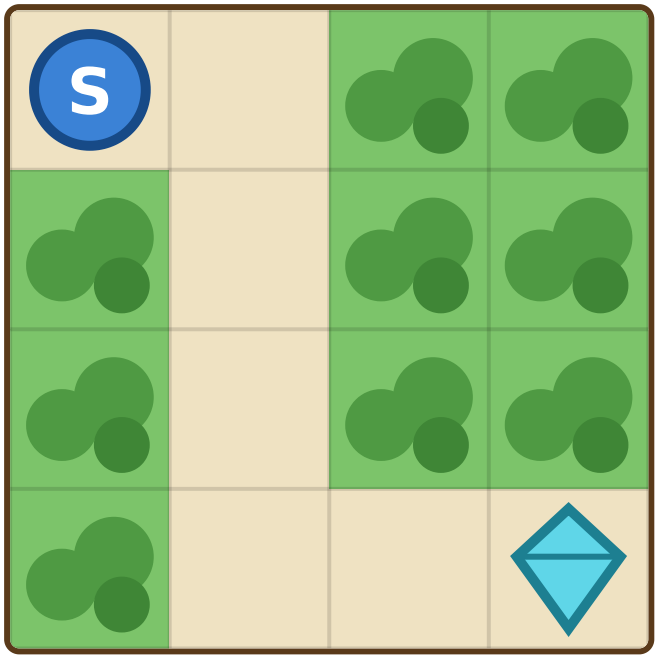
⚠ Common mistake

With 2 arrows you fall short; with 4 you bump into the bushes.

🎬 Video: `videos/en/mazejr-01.mp4`

Stage 2: Go down

Sequence Direction



Problem

Use "right ➡" and "down ⬇".

🎯 **Goal**：Use two kinds of arrows

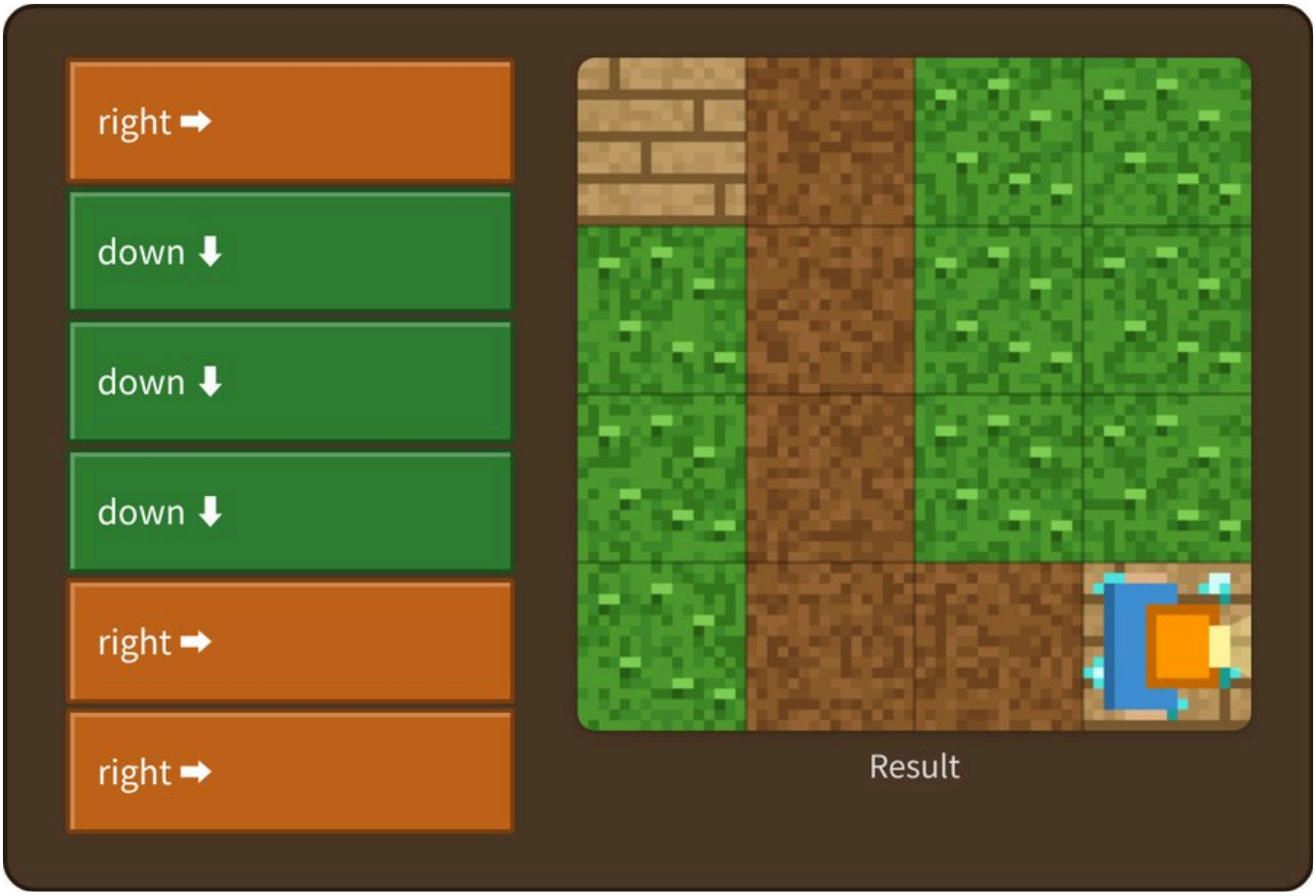
💡 **Hint (look if you get stuck)**

right, then down 3 times, then right 2 times.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Go down



✨ Key point

Right → down → right. Put the arrows in the order you go.

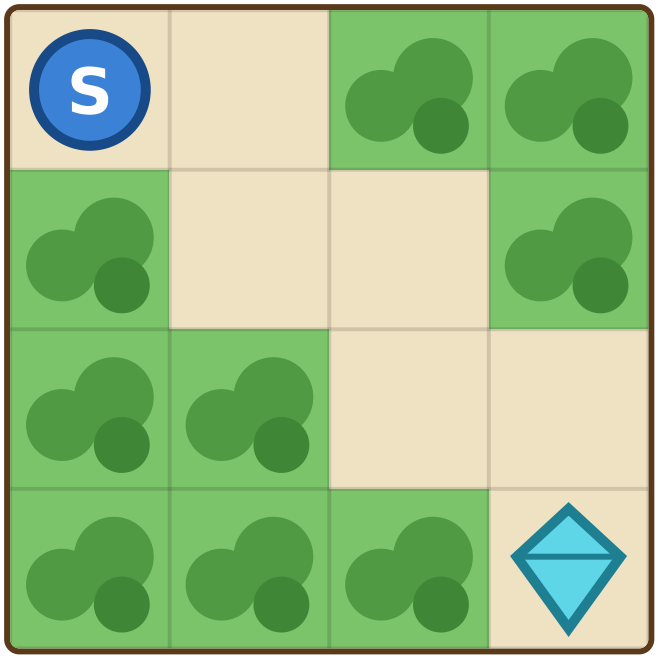
⚠ Common mistake

If you use the wrong number of “down” arrows, you bump into the bushes.

🎬 Video: `videos/en/mazejr-02.mp4`

Stage 3: Stairs

Sequence Patterns



Problem

Right and down, one after the other.

🎯 **Goal**：Find the alternating pattern

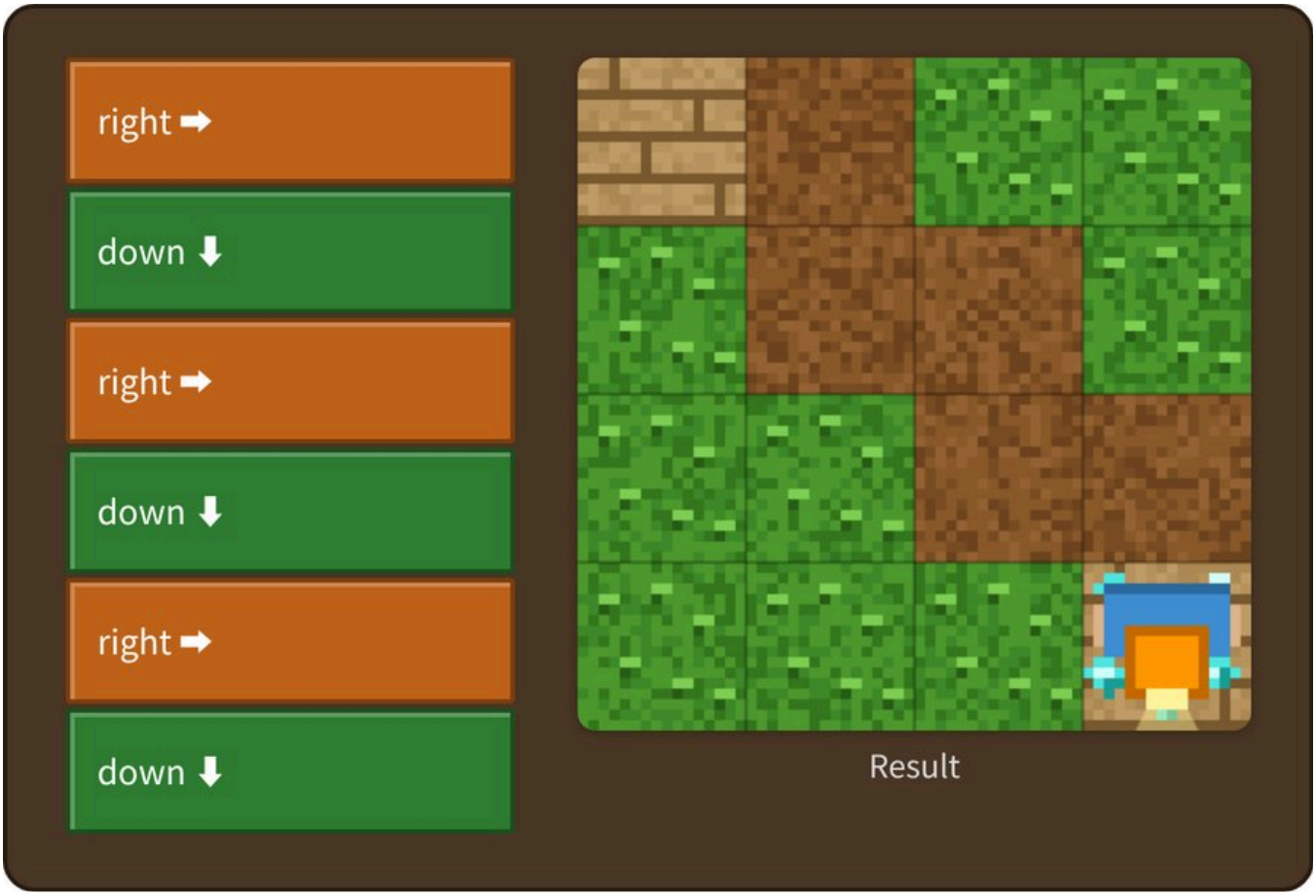
💡 **Hint (look if you get stuck)**

right, down, right, down, right, down

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Stairs



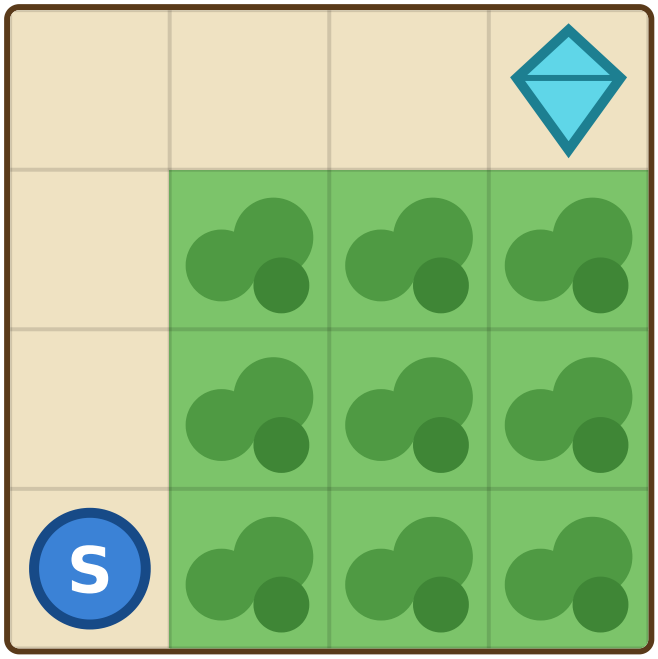
> ✨ **Key point**
>
> Right, down, right, down… the same pair keeps going.

> ⚠ **Common mistake**
>
> Two “right” arrows in a row bump into the bushes.

🎬 Video: `videos/en/mazejr-03.mp4`

Stage 4: Go up

Sequence Direction



Problem

Now use "up ⬆" too.

🎯 **Goal** : Use “up” too

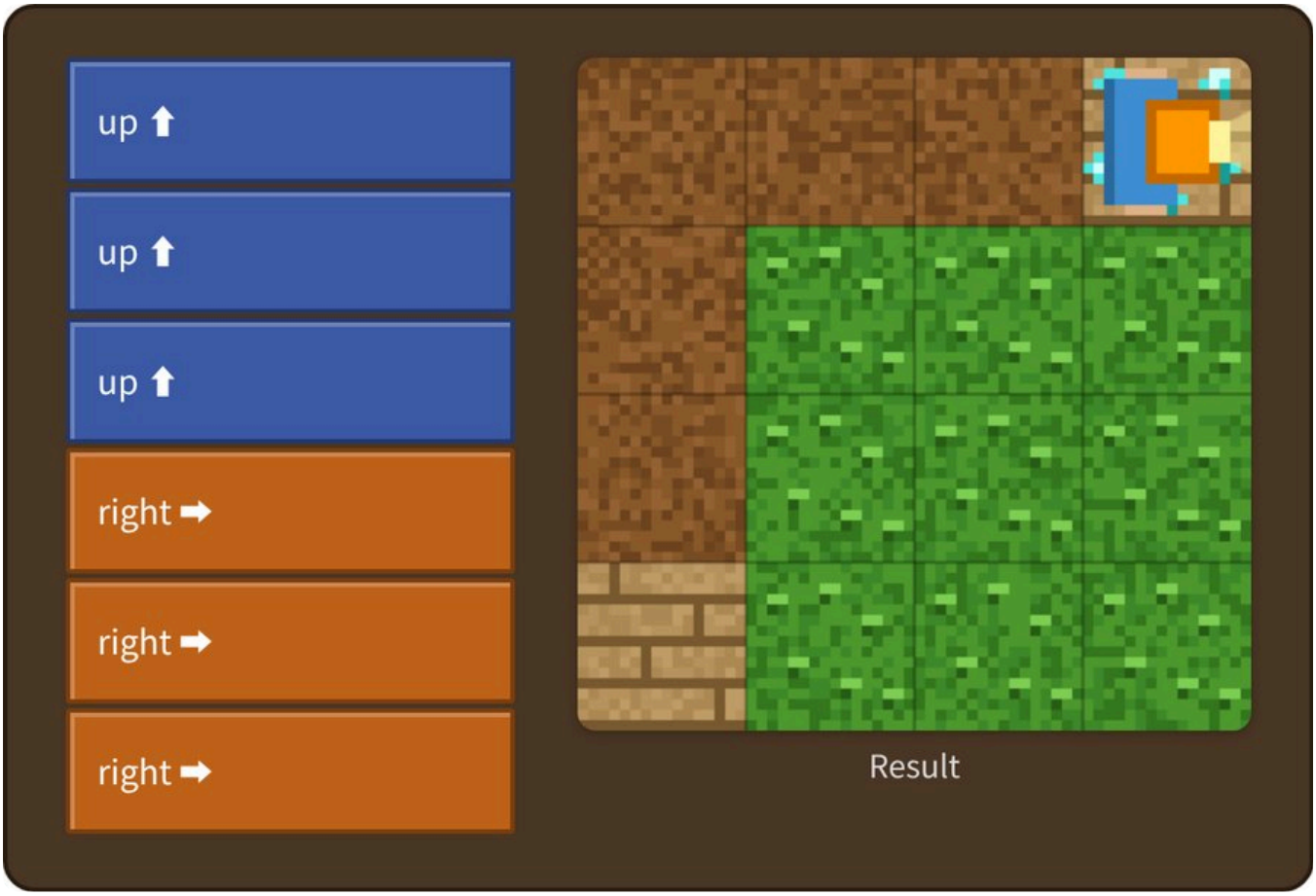
💡 **Hint (look if you get stuck)**

up 3 times, then right 3 times.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Go up



✨ **Key point**

You start at the bottom left. First up 3, then right 3.

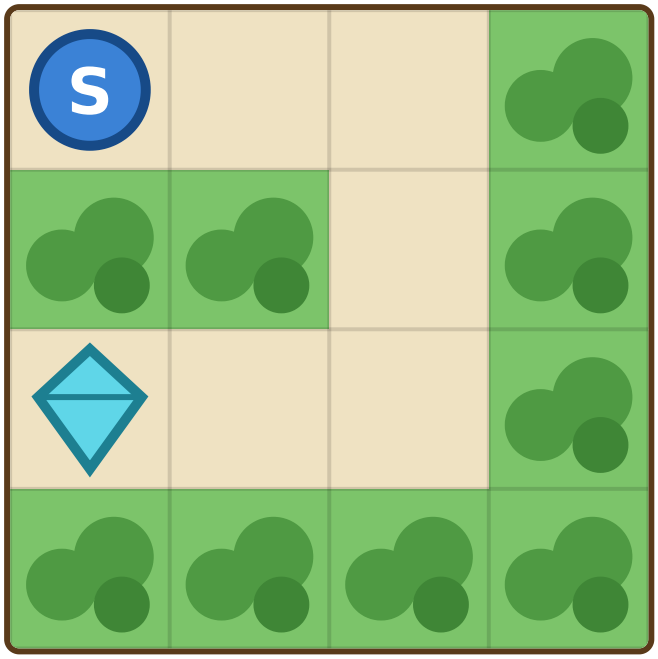
⚠ **Common mistake**

If you start with “right”, you bump into the bushes right away.

🎬 Video: `videos/en/mazejr-04.mp4`

Stage 5: Go around

Sequence Direction



Problem

Use all four arrows.

🎯 **Goal** : Use all four arrows

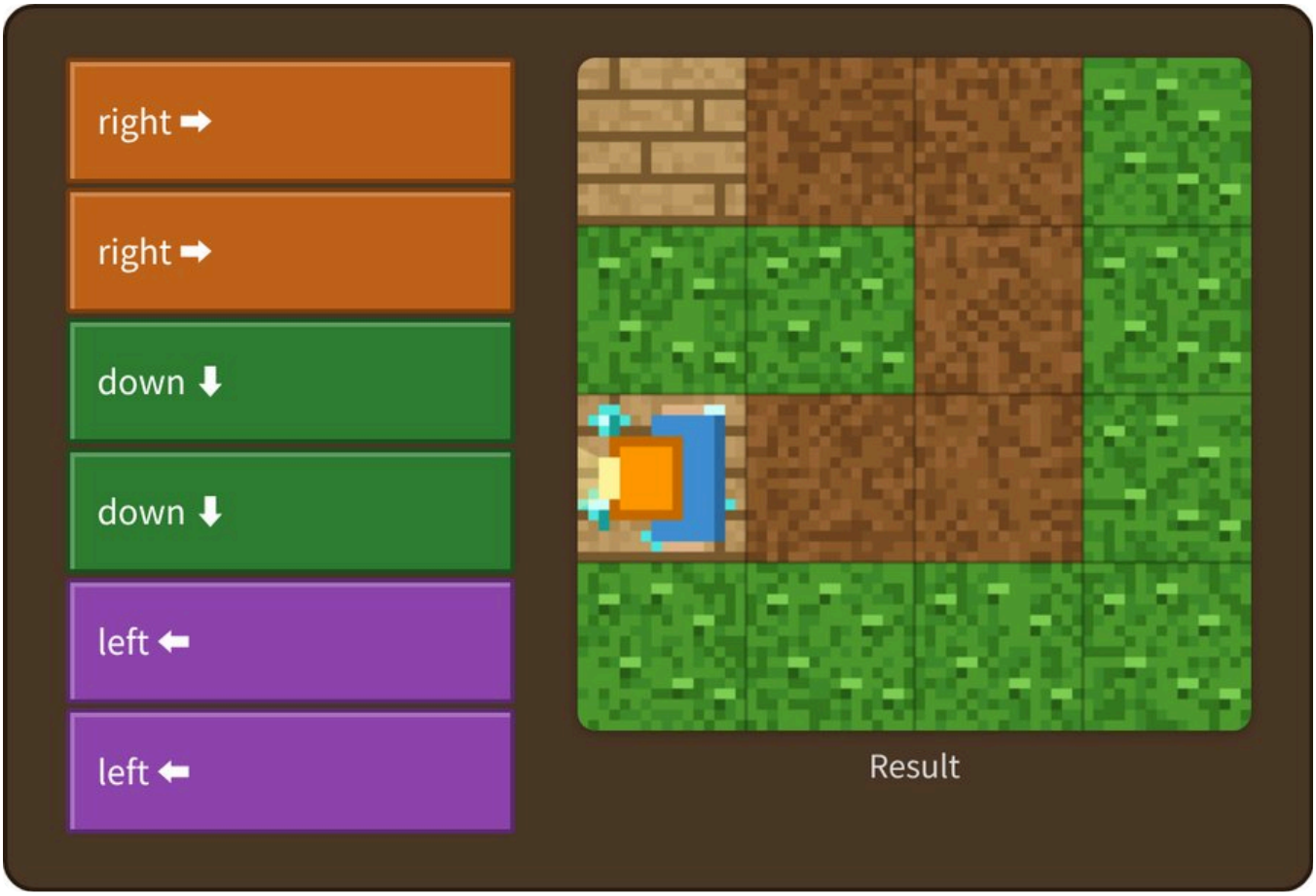
💡 **Hint (look if you get stuck)**

right 2, down 2, left 2.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Go around



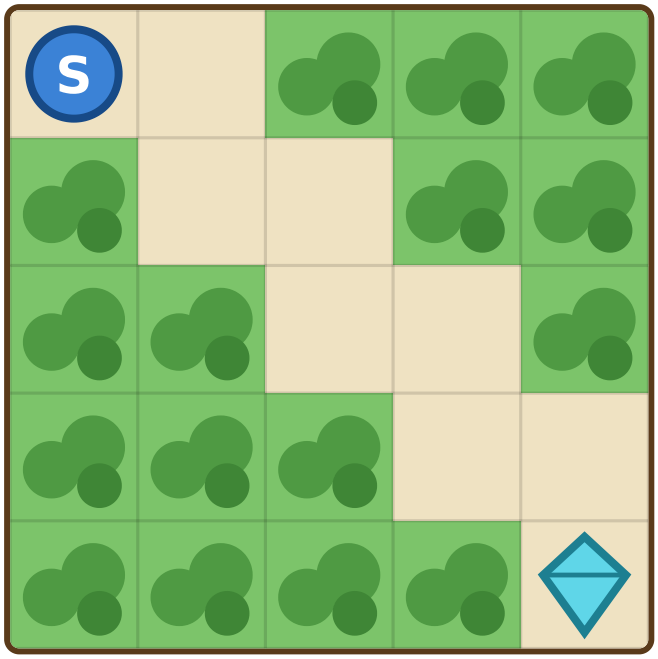
> ✨ **Key point**
>
> Right → down → left. Go around to the diamond.

> ⚠ **Common mistake**
>
> “Left” and “right” are easy to mix up. They mean left and right on the screen.

🎬 Video: `videos/en/mazejr-05.mp4`

Stage 6: Use repeat

Loops Patterns



Problem

A long staircase! Put "right ➡ down ⬇" inside "repeat".

🎯 Target: 3 blocks or fewer

🎯 **Goal**：Make it shorter with repeat

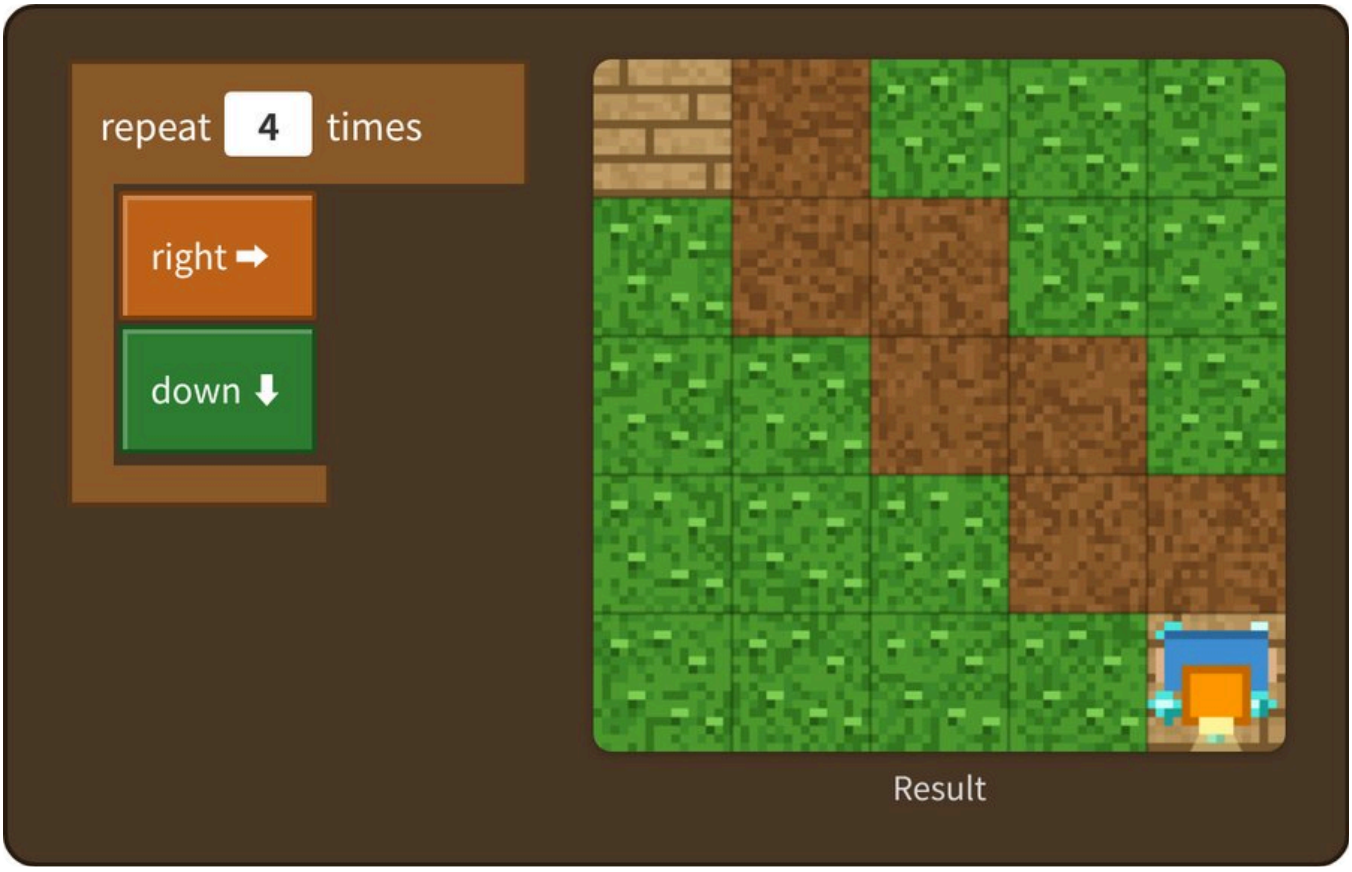
💡 **Hint (look if you get stuck)**

repeat 4 times: right, down

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Use repeat



> ✨ **Key point**
>
> Repeat the “right, down” set 4 times. Just 3 blocks!

> ⚠ **Common mistake**
>
> With 3 repeats you stop just before the diamond.

🎬 Video: `videos/en/mazejr-06.mp4`

Column

Count the squares

In the arrow maze, **counting squares** is the most important thing. Touch each square with your finger and count “1, 2, 3”, then stack that many arrows. When the same arrow repeats, you can pack them into a “repeat” block.

Chapter 3

⛏️ The Block Maze

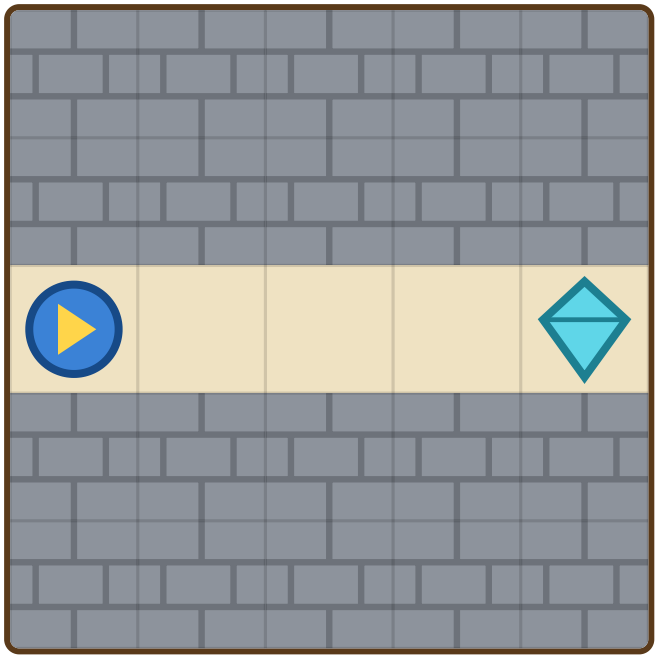
Now the explorer has a **direction**. Use “move forward”, “turn right” and “turn left”. This central chapter goes from **order → loops → if → a way to solve any maze**.

Commands in this chapter

move forward / turn right / turn left / repeat ○ times / repeat until goal / if front (right, left) is open

Stage 1: Go straight

Sequence



Problem

Move the explorer to the diamond 💎. Drag "move forward" blocks from the left and stack them.

🎯 **Goal**：Put the steps in order

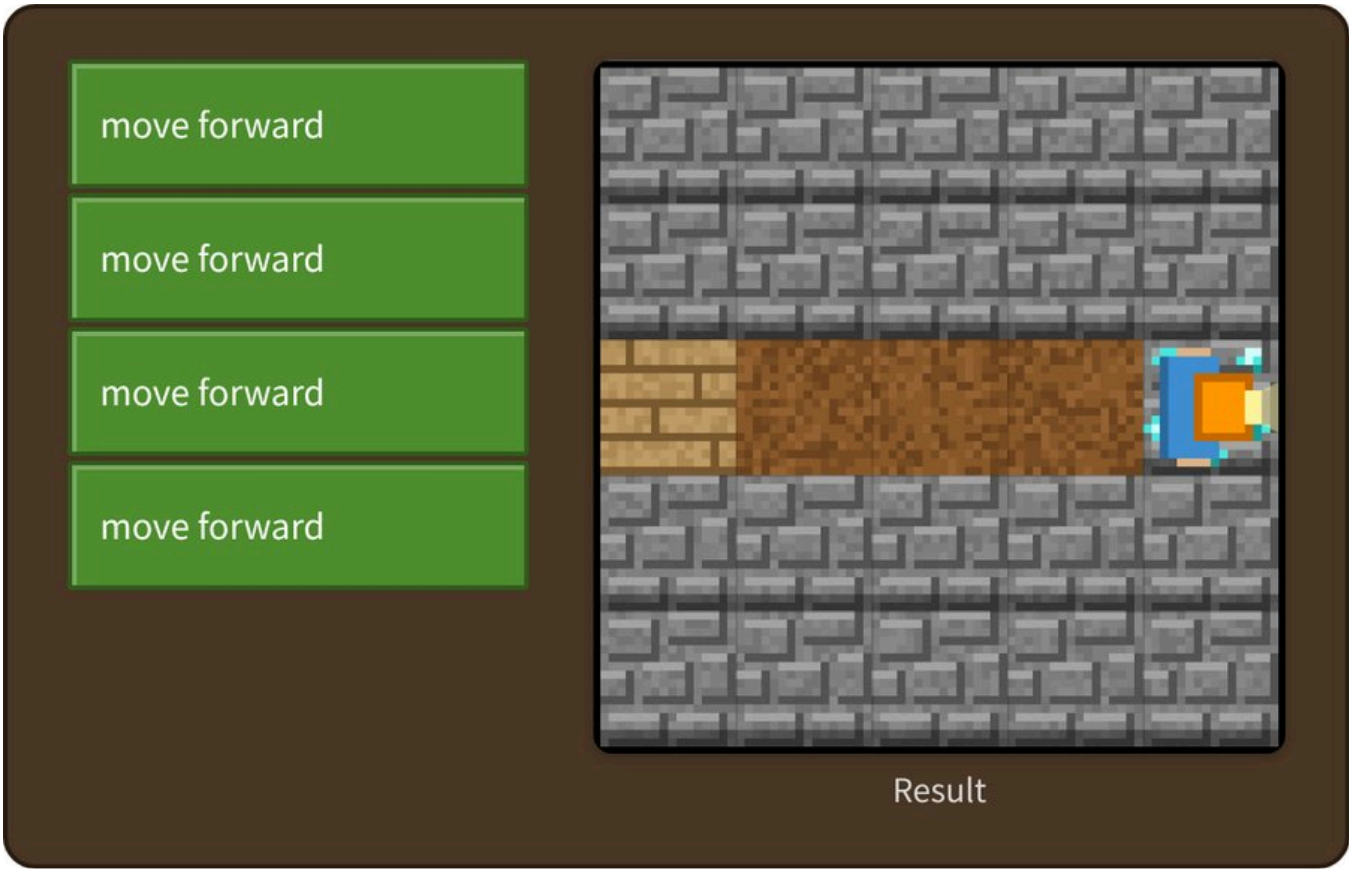
💡 **Hint (look if you get stuck)**

How many squares to the goal? Stack that many "move forward" blocks.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go straight



✨ Key point

Commands run in order from top to bottom. You stacked four “move forward” blocks.

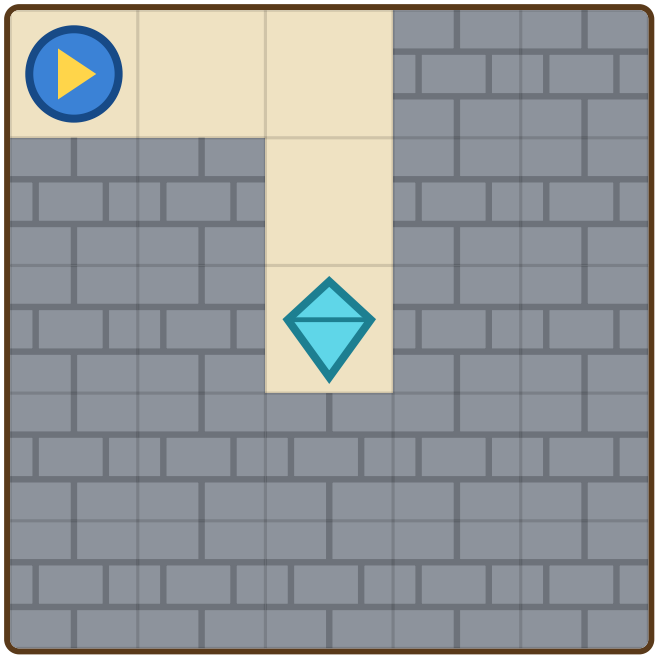
⚠ Common mistake

Blocks that are not connected do not run.

🎬 Video: `videos/en/maze-01.mp4`

Stage 2: Let's turn

Sequence Direction



Problem

You turn on the way. "Forward" is where the explorer's light (yellow beam) is shining.

🎯 **Goal**：Think about direction

💡 **Hint (look if you get stuck)**

After 2 squares, "turn right". Turning does not move you to another square.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Let's turn



✨ Key point

“Turn” only changes the direction. The explorer stays on the same square.

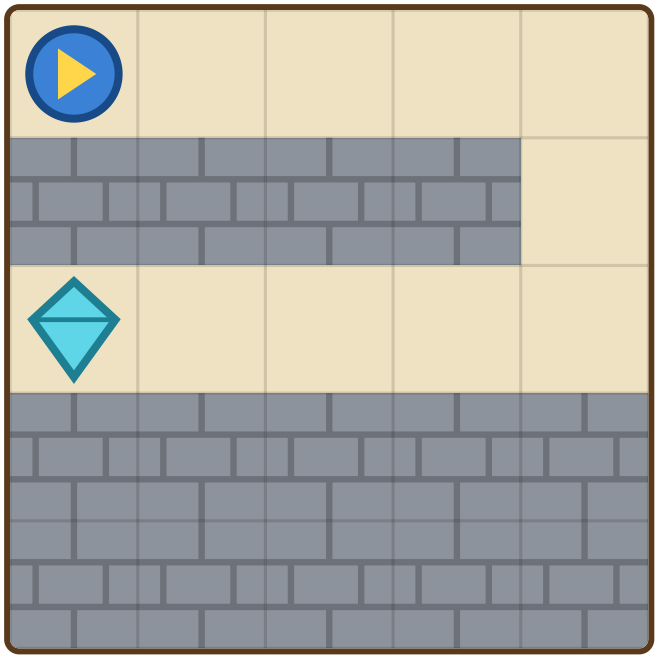
⚠ Common mistake

Turning alone does not move you. Add “move forward” after turning.

🎬 Video: `videos/en/maze-02.mp4`

Stage 3: A long path

Sequence Direction



Problem

Walk the long path. Pretend you are the explorer and think about which way to turn.

🎯 **Goal**：Build a long set of steps

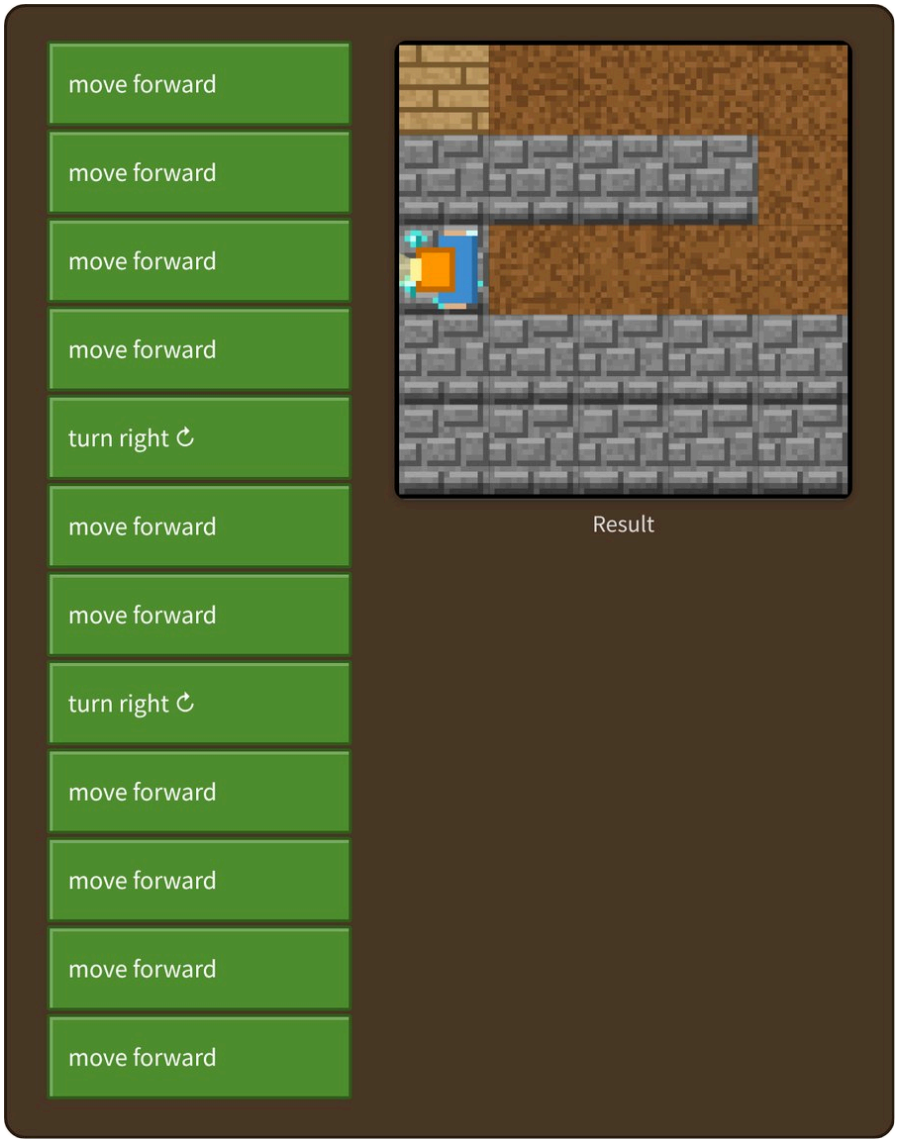
💡 **Hint (look if you get stuck)**

Forward 4 → turn right → forward 2 → turn right → forward 4

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: A long path



✨ Key point

That took 12 blocks! Next, you will learn a way to make it shorter.

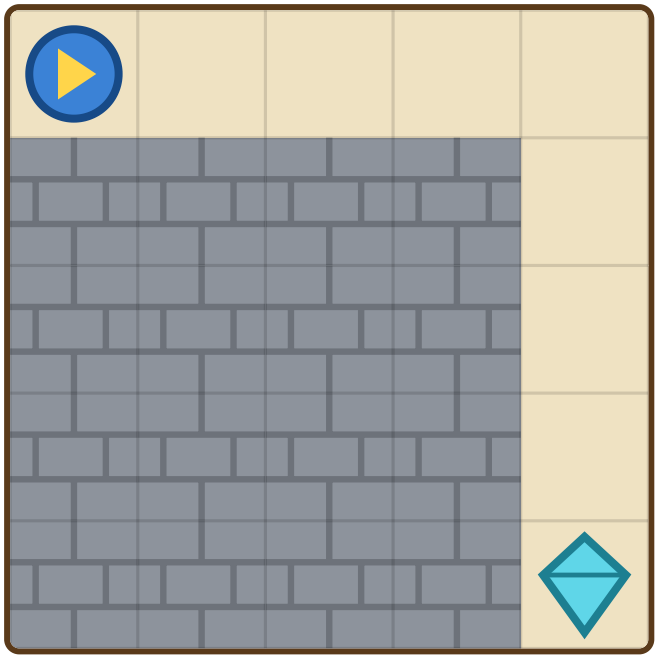
⚠ Common mistake

It is easy to mix up left and right. Imagine you are the explorer.

🎬 Video: `videos/en/maze-03.mp4`

Stage 4: Use a loop

Loops



Problem

Stacking the same block again and again is hard work. Put blocks inside "repeat" to repeat them.

🎯 Target: 5 blocks or fewer

🎯 **Goal**：Make it shorter with a loop

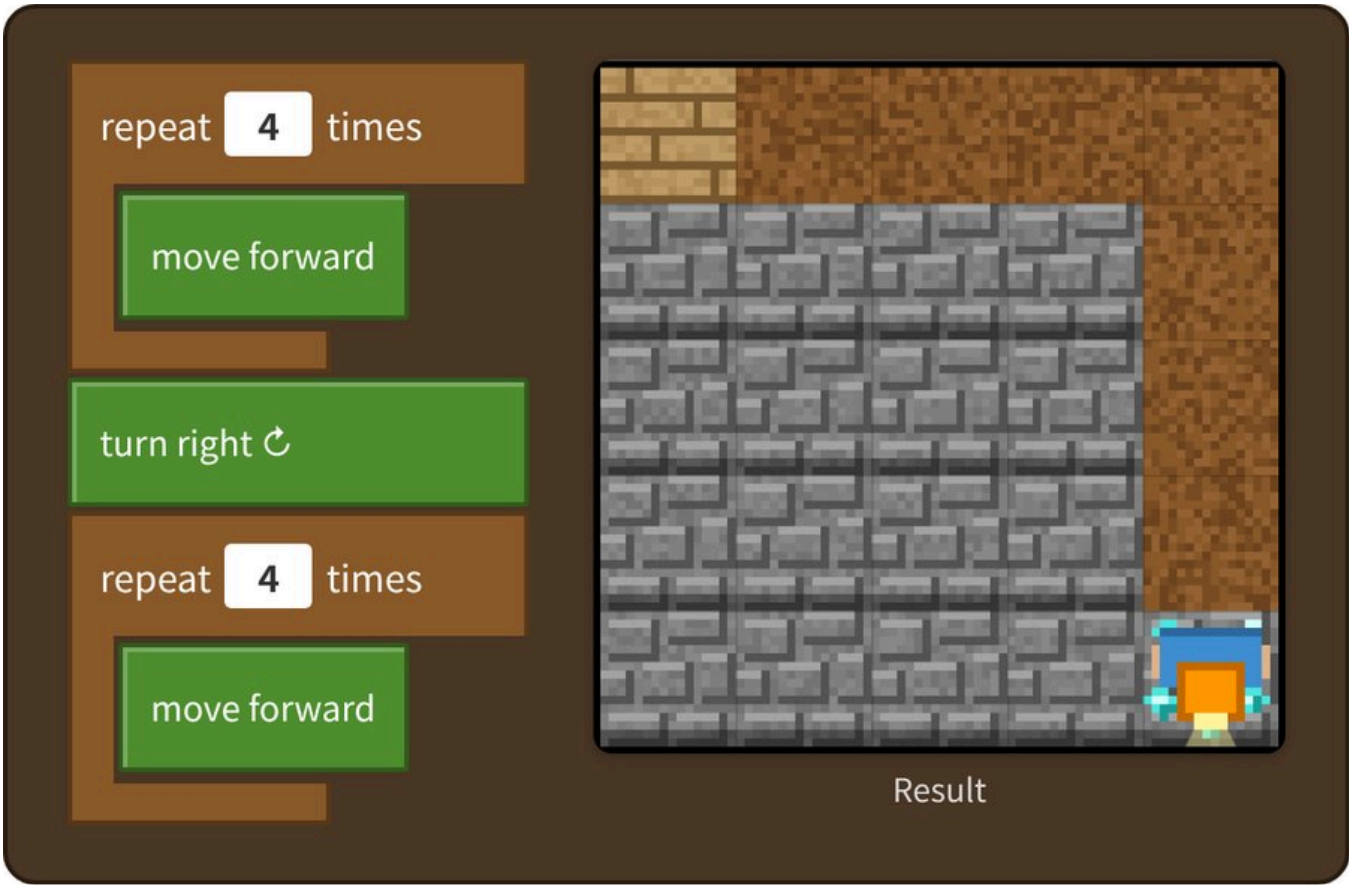
💡 **Hint (look if you get stuck)**

"repeat 4 (move forward)" → "turn right" → "repeat 4 (move forward)"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Use a loop



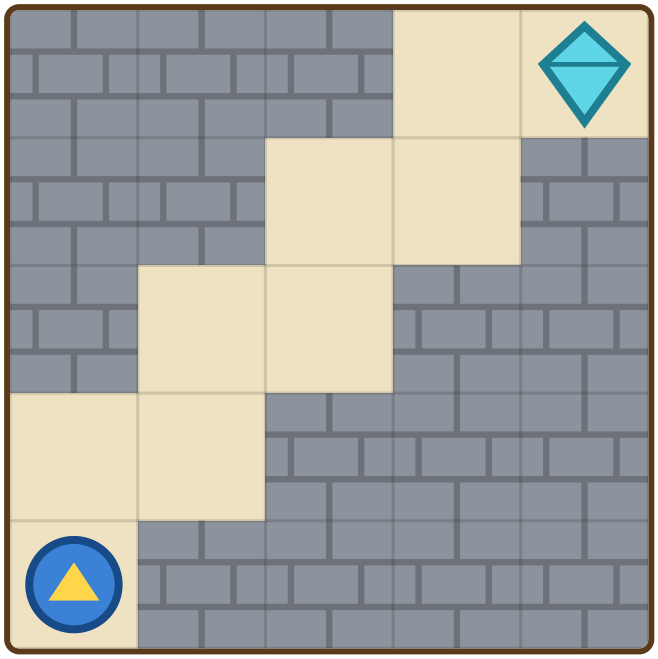
> ✨ **Key point**
>
> Repeated blocks can be grouped with “repeat __ times”.

> ⚠ **Common mistake**
>
> Putting “turn right” inside the loop changes the movement.

🎬 Video: `videos/en/maze-04.mp4`

Stage 5: Climb the stairs

Loops Patterns



Problem

Look closely at the stairs. The same set of moves repeats.

🎯 Target: 5 blocks or fewer

🎯 **Goal**：Find the pattern that repeats

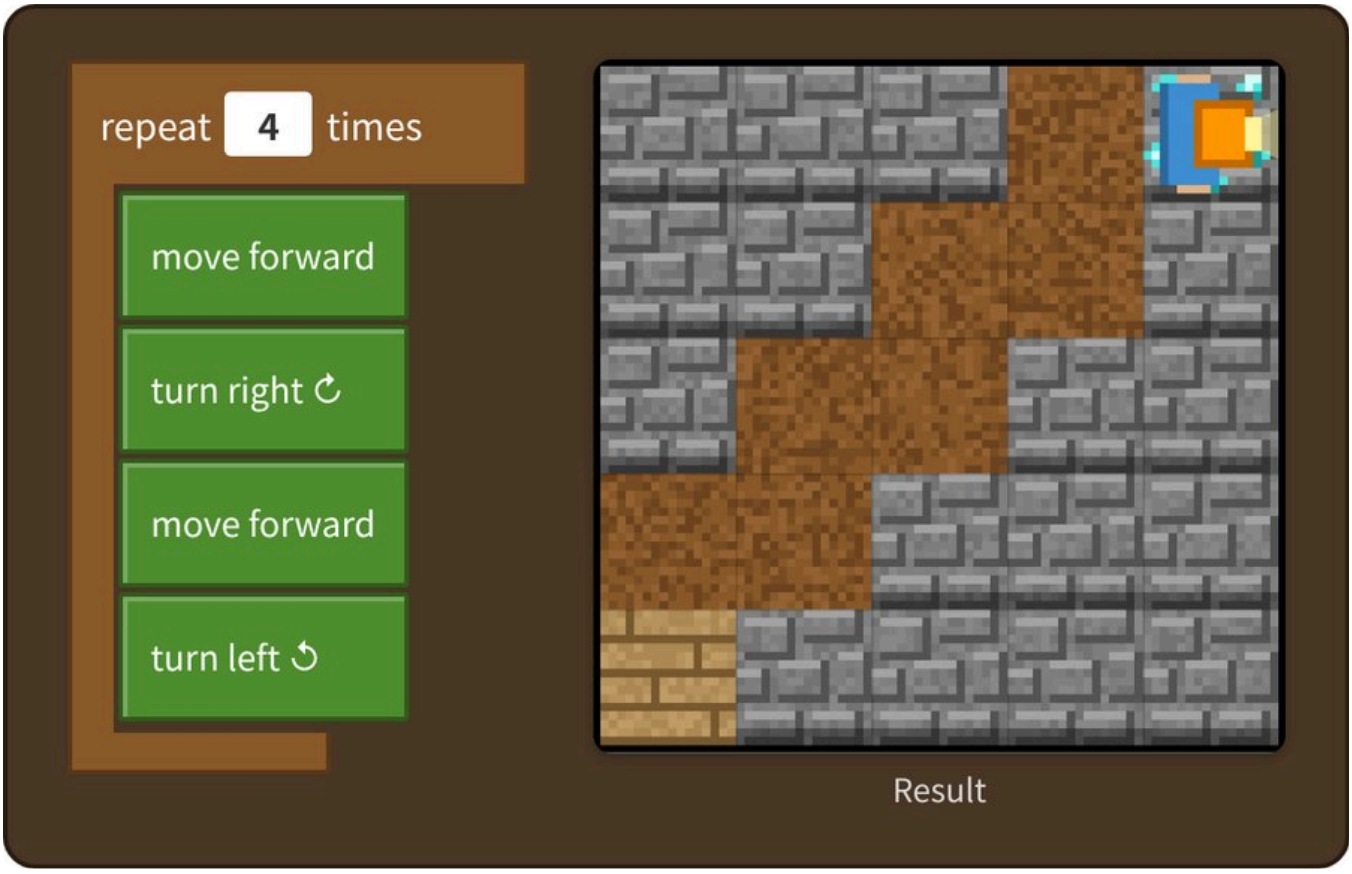
💡 **Hint (look if you get stuck)**

One set is "forward → turn right → forward → turn left". How many times should it repeat?

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Climb the stairs



✨ Key point

One set is forward, right, forward, left. The program stops when it reaches the goal.

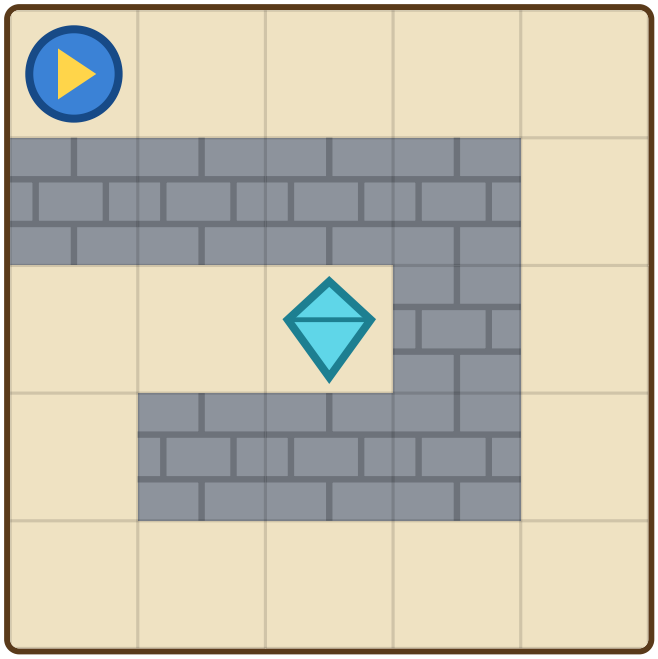
⚠ Common mistake

It is easy to get the order of the set wrong (forward → right → forward → left).

🎬 Video: `videos/en/maze-05.mp4`

Stage 6: If…

Conditions | Repeat until



Problem

Try "repeat until goal" and "if front is open". The explorer looks ahead and decides to move or turn by itself.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Let the explorer decide with “if”

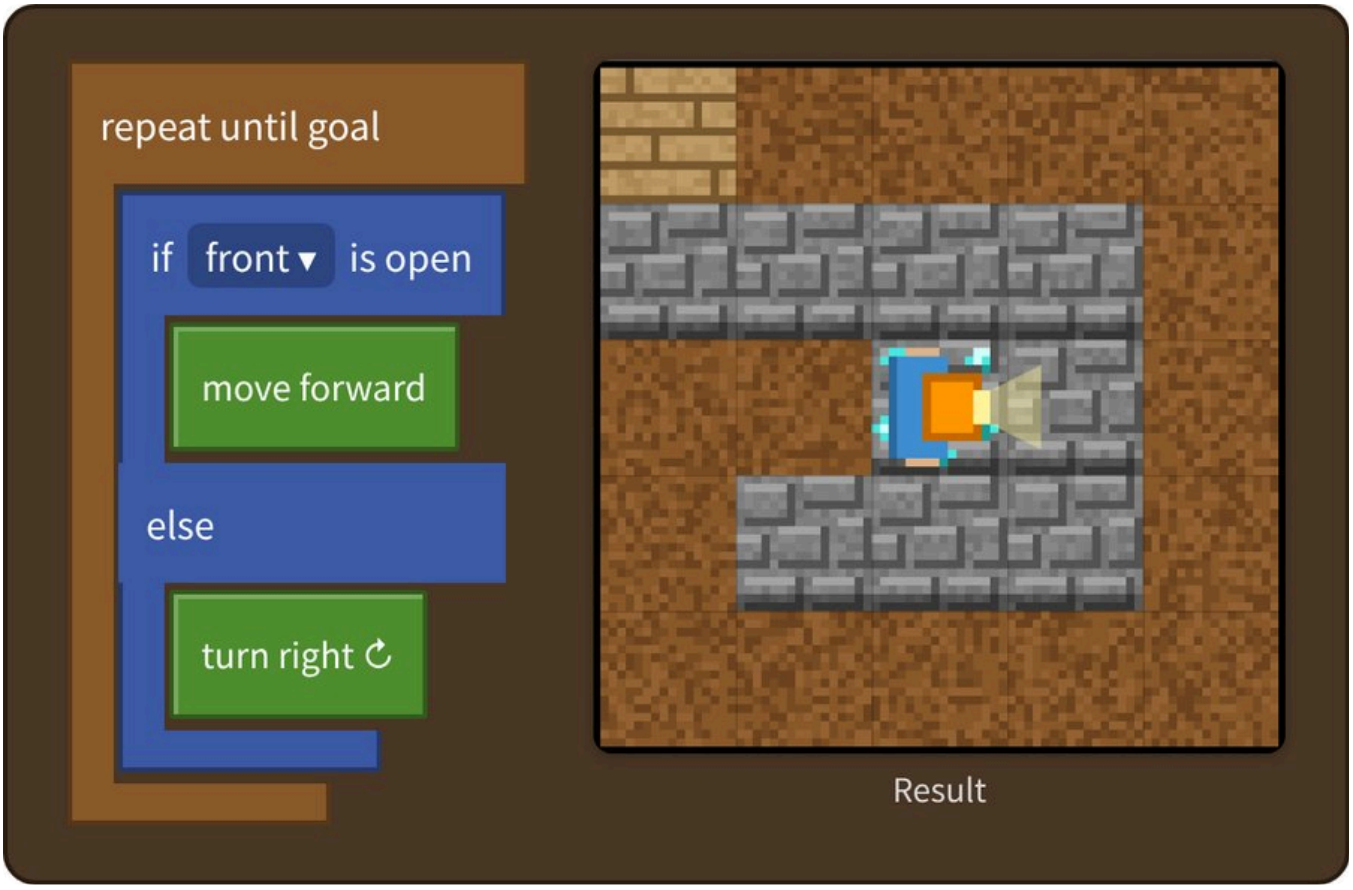
💡 **Hint (look if you get stuck)**

Repeat until goal: if front is open "move forward", else "turn right".

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: If…



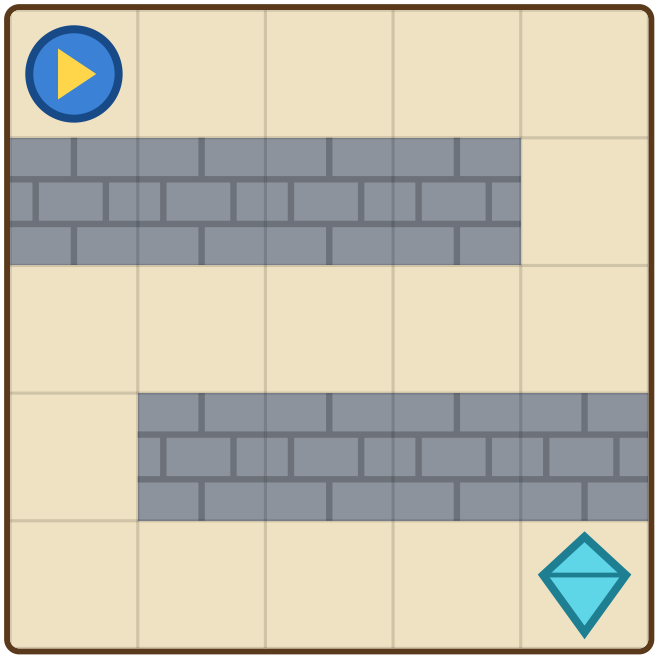
> ✨ **Key point**
>
> If the front is open, move. If not, turn right. The explorer decided by itself!

> ⚠ **Common mistake**
>
> If the “if” is outside the loop, it only checks once.

🎬 Video: `videos/en/maze-06.mp4`

Stage 7: Right? Left?

Conditions Nesting



Problem

Now you turn both right and left. If the front is blocked, check the right. If that is blocked too, turn left.

🎯 Target: 6 blocks or fewer

🎯 **Goal**：Put an “if” inside an “if”

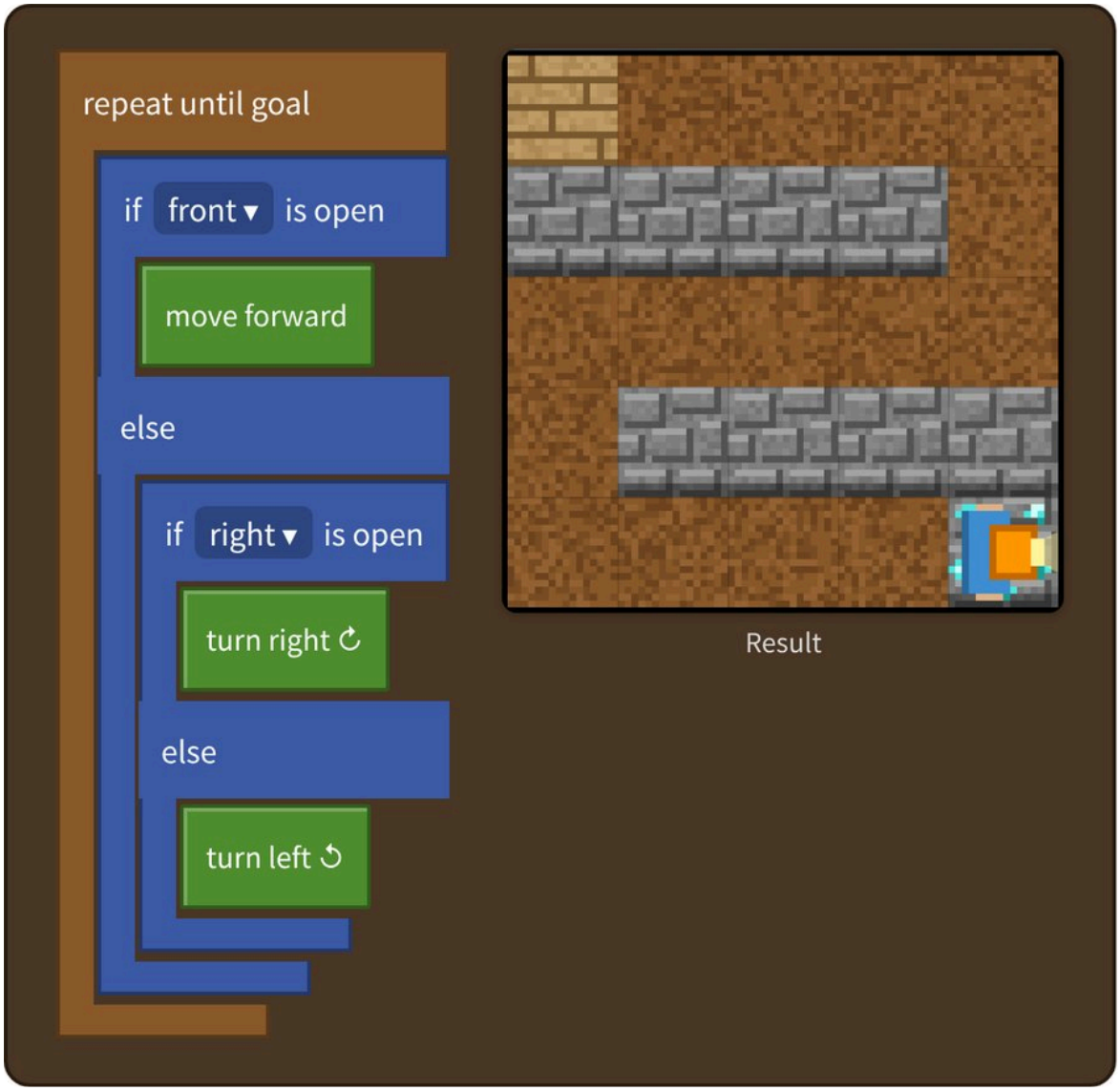
💡 **Hint (look if you get stuck)**

Put another "if right is open" inside "else".

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 7: Right? Left?



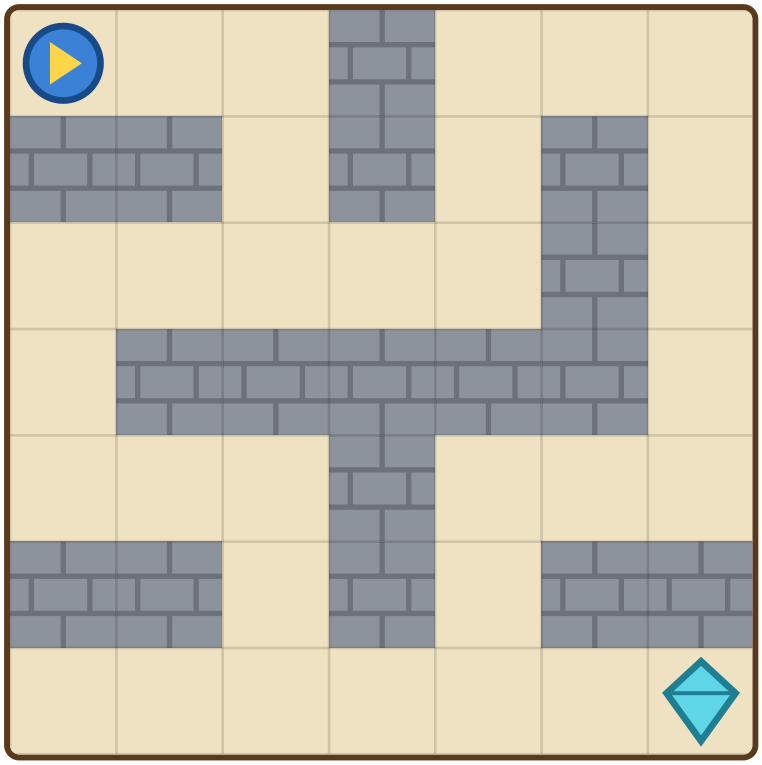
> ✨ **Key point**
>
> Another “if” inside “else” lets it choose right or left.

> ⚠ **Common mistake**
>
> With the Stage 6 program, you hit a dead end after turning.

🎬 Video: `videos/en/maze-07.mp4`

Stage 8: Any maze!

Generalization | Conditions | Repeat until



Example: the maze changes every time you run

Problem

Final challenge! Your program is tested on 30 random mazes. Reach the goal in all of them to clear!

🎯 **Goal**：A way to solve any maze

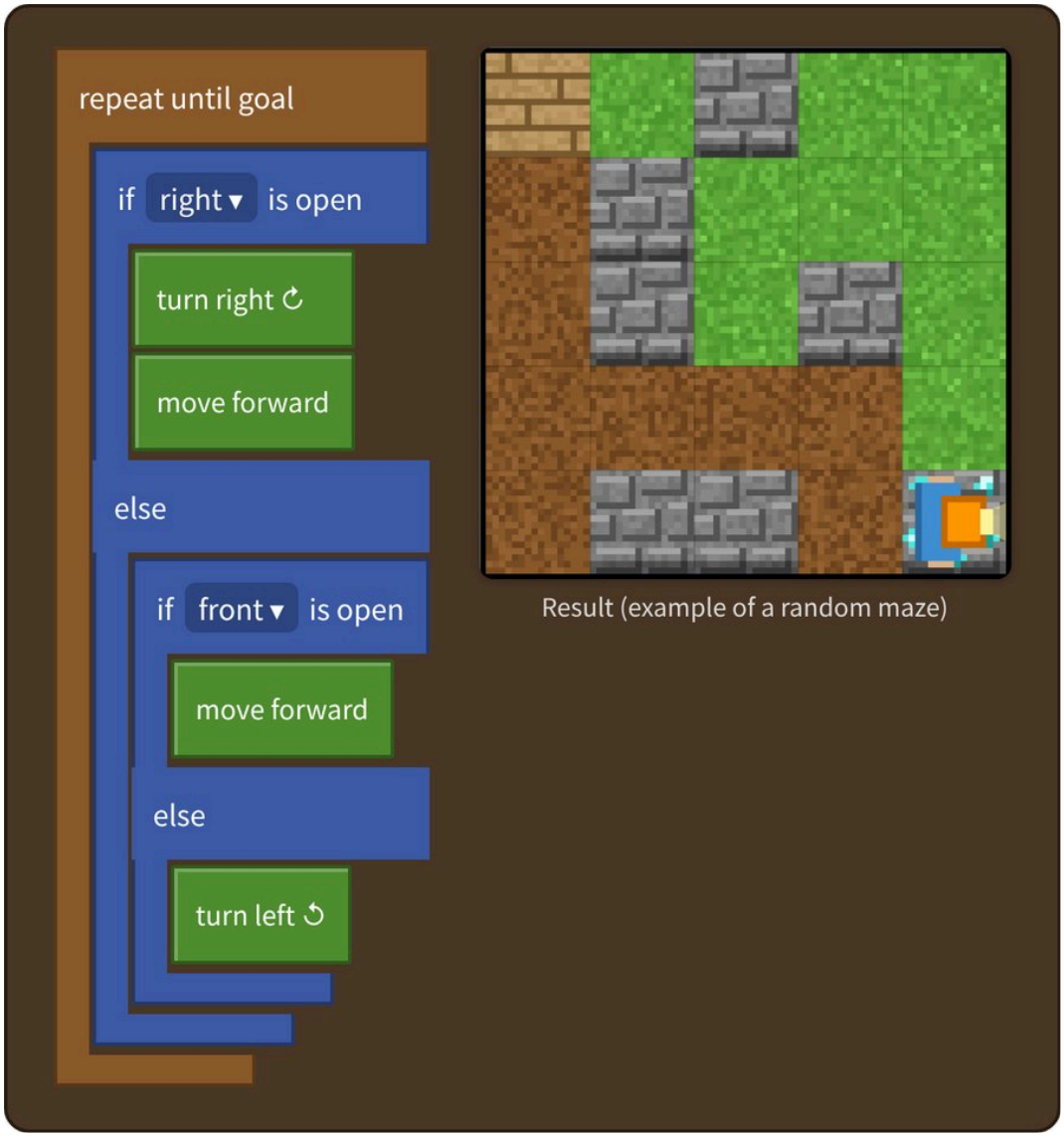
💡 **Hint (look if you get stuck)**

The "right-hand rule": keep your right hand on the wall and you always reach the goal. If the right is open, turn right first, then move.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 8: Any maze!



Result (example of a random maze)

> ✨ **Key point**
>
> The “right-hand rule”: keep your right hand on the wall, and you can solve any maze!

> ⚠ **Common mistake**
>
> The Stage 7 program can keep going in circles in some mazes.

🎬 Video: `videos/en/maze-08.mp4`

Column

The right-hand rule

Even in a dark maze, if you walk **keeping your right hand on the wall**, you will always reach the exit (when all the walls are connected). This is the “right-hand rule”.

As a program: **repeat until goal** — if the right is open, turn right and move; else if the front is open, move; otherwise turn left. Stage 8 can be solved this way. A method that always works is called an **algorithm**.

Chapter 4

🗝️ The Keys & Doors Maze

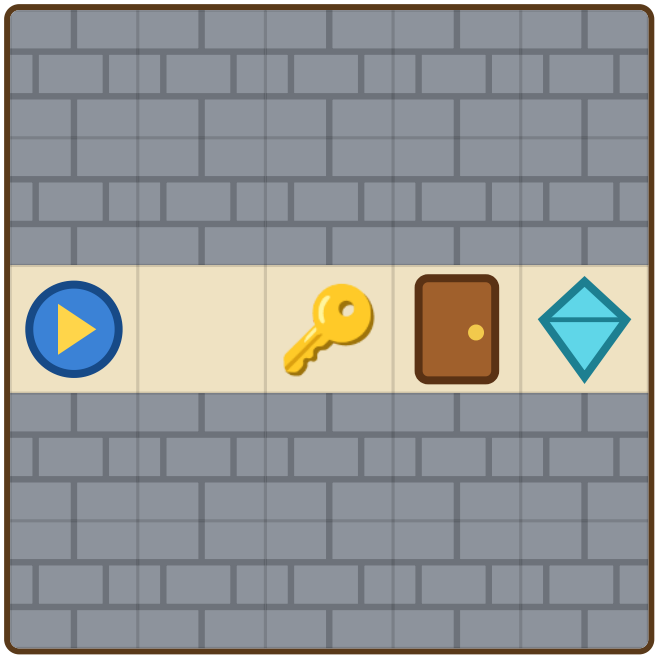
Pick up keys 🔑 and open doors 🚪 as you go. You must remember “how many keys am I carrying now?” This is called **state**.

Commands in this chapter

move forward / turn right, left / pick up key / open door / repeat / if key here / if door ahead

Stage 1: Pick up the key, open the door

Sequence State



Problem

Move onto the key 🔑 and use "pick up key". In front of the door 🚪, use "open door" and you can go through.

🎯 **Goal**：Pick up the key and open the door

💡 **Hint (look if you get stuck)**

forward → forward → pick up key → open door → forward → forward

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Pick up the key, open the door



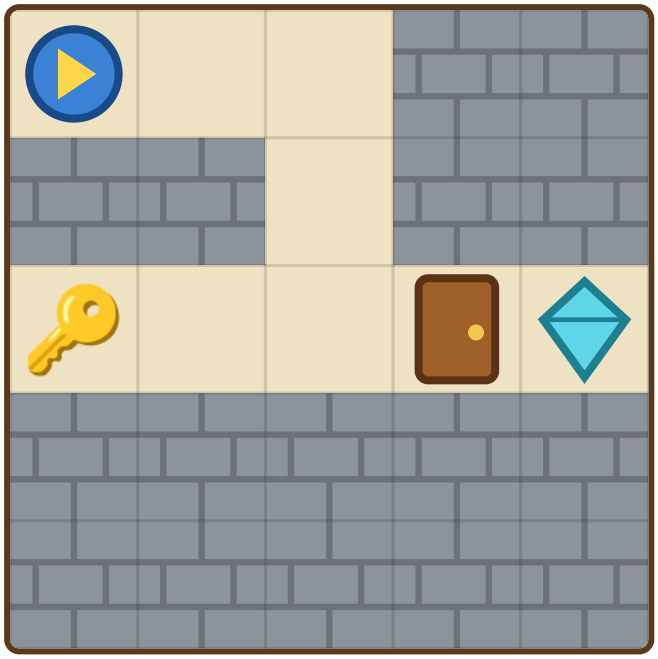
> ✨ **Key point**
>
> “Pick up key” makes your key count 1, and “open door” brings it back to 0.

> ⚠ **Common mistake**
>
> Picking up when you are not on the key gives “There is no key here.” Check how far you move.

🎬 Video: `videos/en/mazekey-01.mp4`

Stage 2: Where is the key?

Sequence State Decomposition



Problem

The diamond is behind the door. Go and get the key first. Without a key, the door will not open.

🎯 **Goal**：Get the key first

💡 **Hint (look if you get stuck)**

Walk to the key and pick it up. Then turn right twice and go back.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Where is the key?



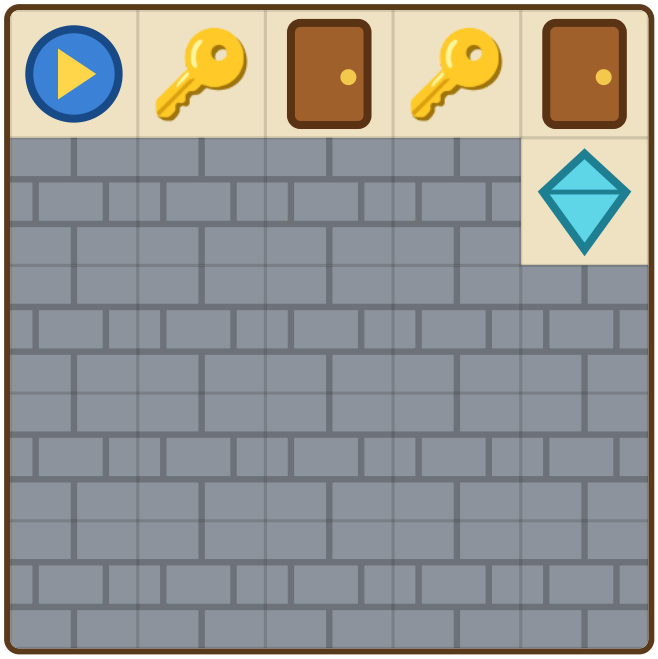
> ✨ **Key point**
>
> Take a side trip before the goal. Think of it as “get the key” and “go back to the door”.

> ⚠ **Common mistake**
>
> Going to the door without a key gives “The door is locked. Where is the key?”

🎬 Video: `videos/en/mazekey-02.mp4`

Stage 3: Two doors

Loops Patterns State



Problem

Keys and doors are in a row. Find the set of moves that repeats and use "repeat".

🎯 Target: 7 blocks or fewer

🎯 **Goal**：Repeat the same set

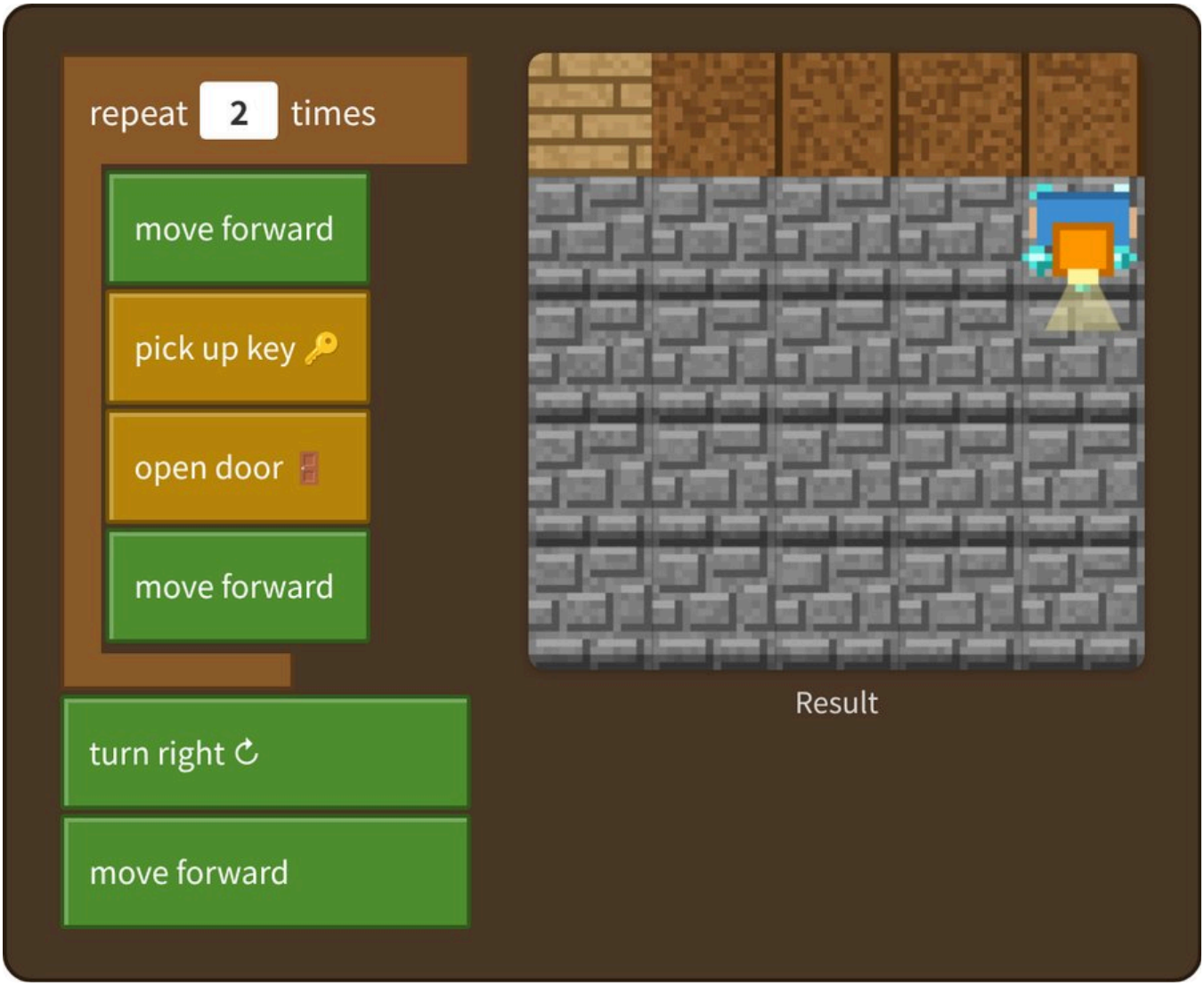
💡 **Hint (look if you get stuck)**

Repeat 2 times: forward → pick up key → open door → forward. Then turn right and move forward.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 3: Two doors



✨ Key point

Key → door appears twice. Repeat the 4 blocks 2 times.

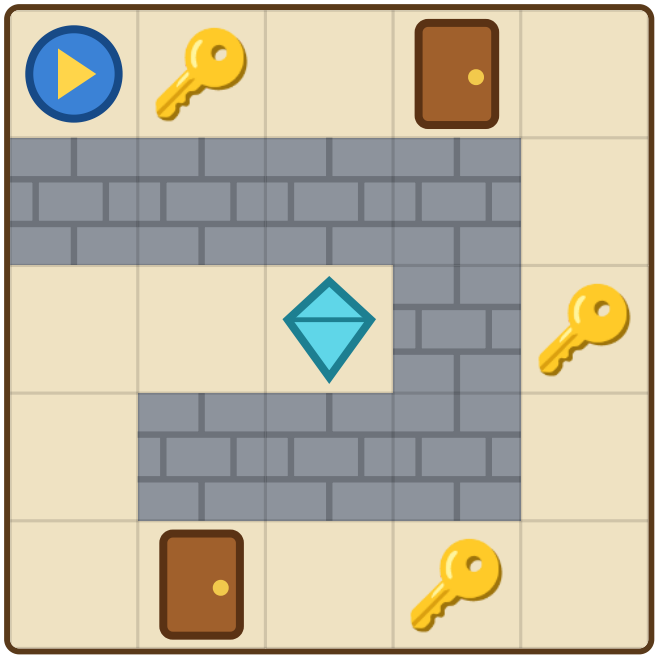
⚠ Common mistake

If you forget “move forward” after “open door”, you never reach the next key.

🎬 Video: `videos/en/mazekey-03.mp4`

Stage 4: A thinking explorer

Conditions | Repeat until | State | Generalization



Problem

Use "repeat until goal" and "if". Make a program that picks up a key when it finds one and opens a door when there is one.

🎯 Target: 8 blocks or fewer

🎯 **Goal**：Check and decide by yourself

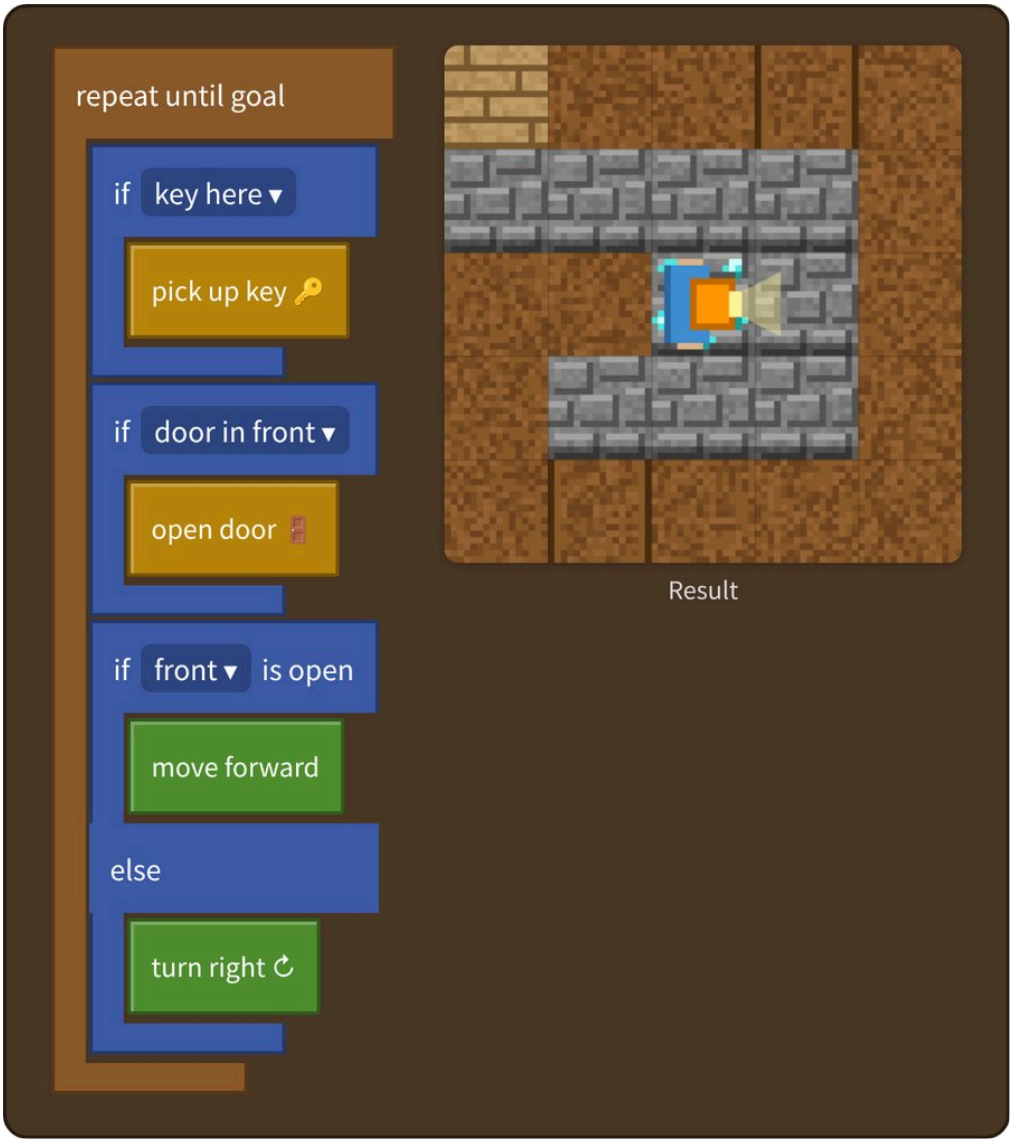
💡 **Hint (look if you get stuck)**

Inside the loop: if key here → pick up key; if door in front → open door; if front is open → forward, else → turn right.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: A thinking explorer



> ✨ **Key point**
>
> Pick up a key when you find one, open a door when there is one, and then go forward or turn right. Check all three every time.

> ⚠ **Common mistake**
>
> If “if front is open” comes before “if door in front”, you turn around at the door.

🎬 Video: `videos/en/mazekey-04.mp4`

Column

Write down what you carry

Picking up a key adds 1 to what you carry; opening a door takes 1 away. It is easy to forget if you only think in your head, so write it on the worksheet: “forward → 1 key → open → 0 keys”. Computers also remember this kind of “state” in **variables**.

Chapter 5

👾 The Dodge-the-Slime Maze

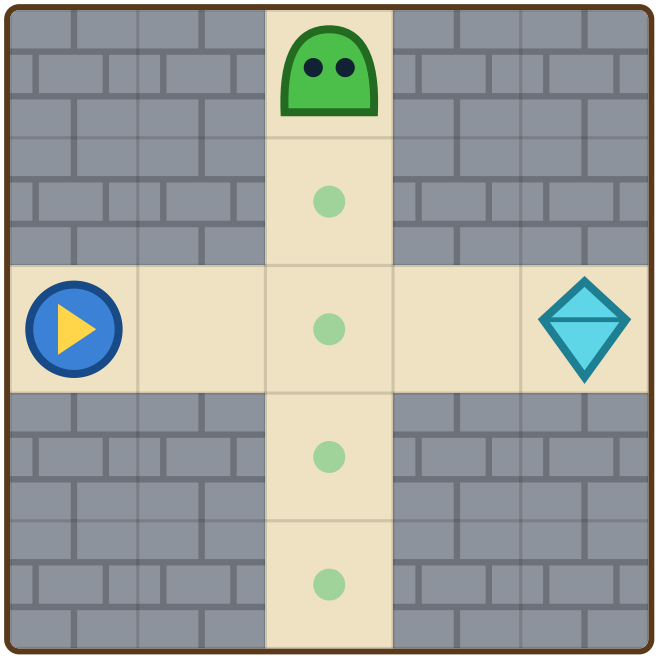
Slimes go back and forth on fixed tracks. Every time the explorer acts (move, turn or wait), each slime moves one square. Use “wait” to get the **timing** right.

Commands in this chapter

move forward / turn right, left / wait / repeat until goal / if danger ahead / if front is open

Stage 1: Wait and dodge

Sequence Timing



Problem

The slime goes up and down. Use "wait" to time your move so you do not bump into it.

🎯 **Goal**：Wait to time your move

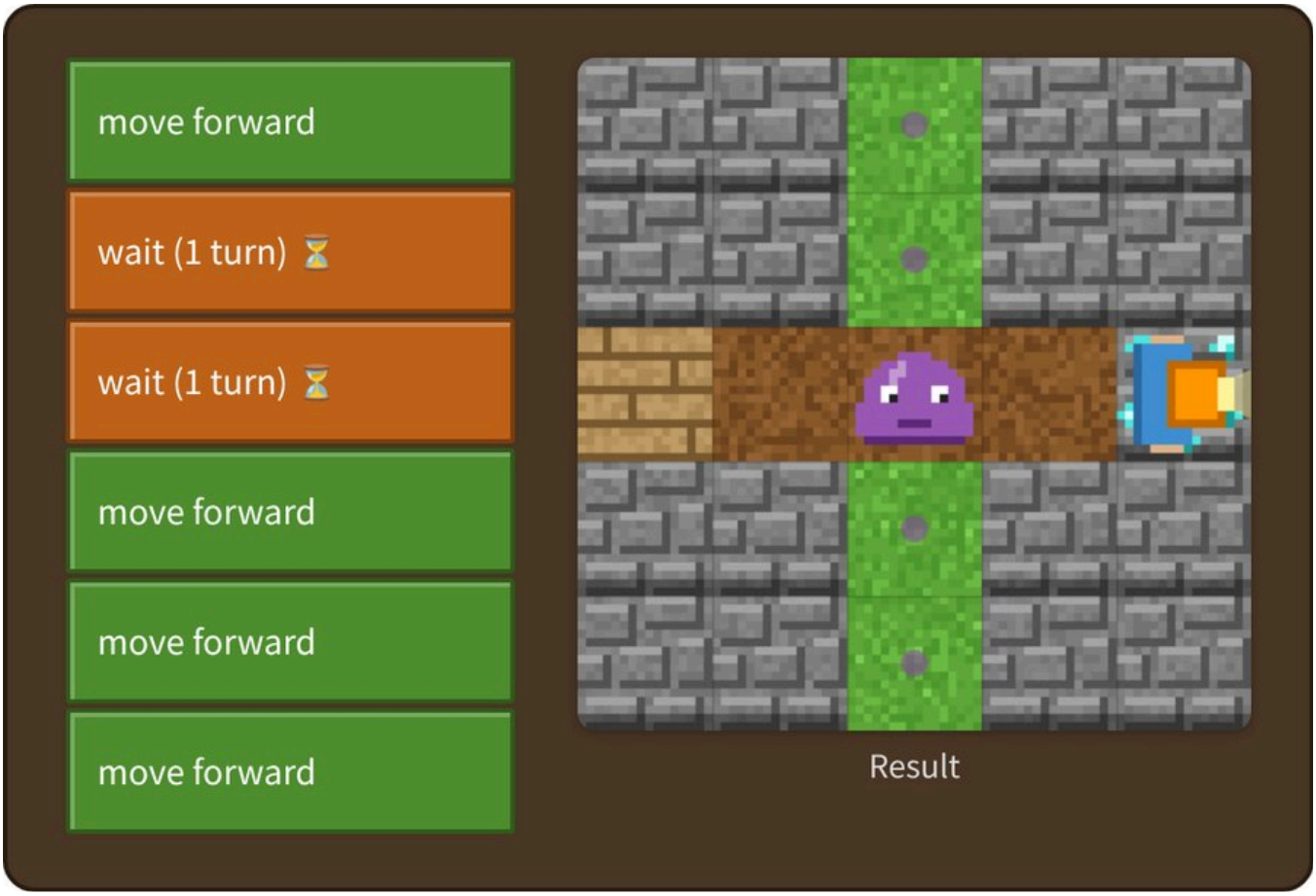
💡 **Hint (look if you get stuck)**

Move 1 square, then wait 2 times until the slime passes. Then move 3 squares.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Wait and dodge



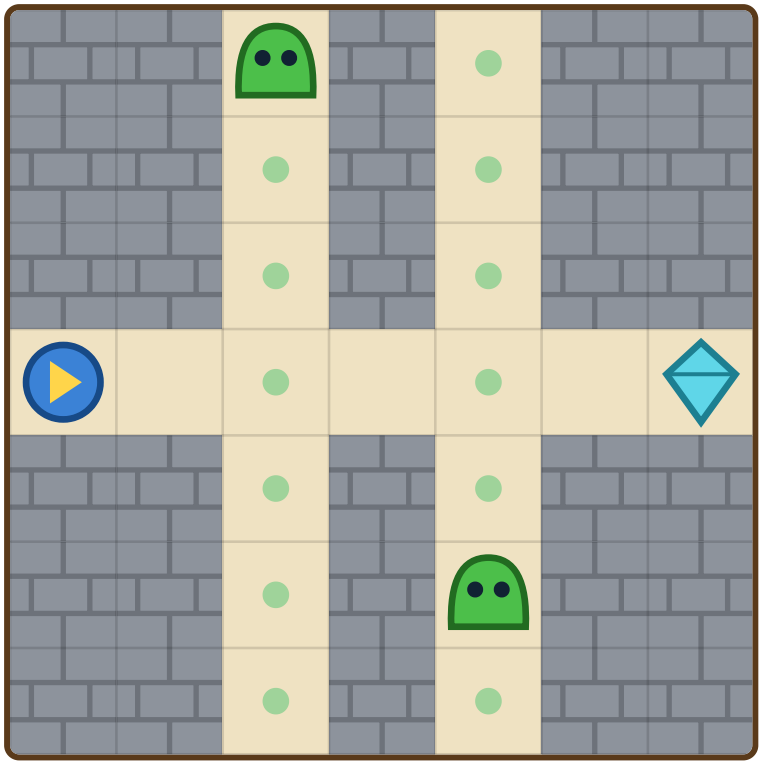
> ✨ **Key point**
>
> The slime moves one square each turn. Wait 2 times until it passes.

> ⚠ **Common mistake**
>
> With only one “wait”, you walk right into the slime.

🎬 Video: `videos/en/mazeenemy-01.mp4`

Stage 2: Wait if it is dangerous

Conditions | Repeat until | Timing



Problem

Use "if danger ahead". It is dangerous when a slime is in front of you, or will move in front of you next.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Wait when it is dangerous

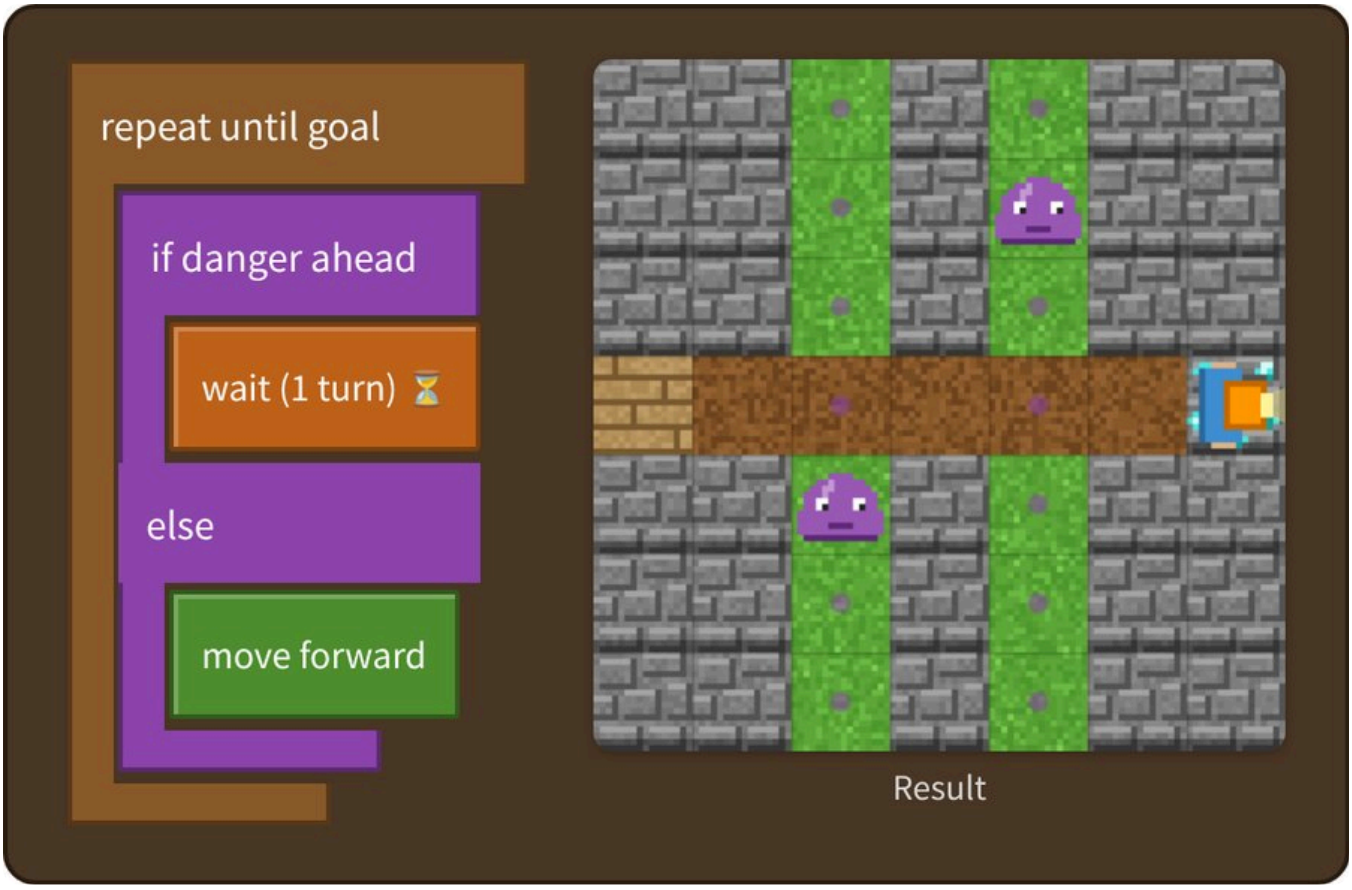
💡 **Hint (look if you get stuck)**

Repeat until goal: if danger ahead → wait, else → move forward.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Wait if it is dangerous



✨ **Key point**

“Danger” means a slime is in front or will come there next. If so, wait; if not, move.

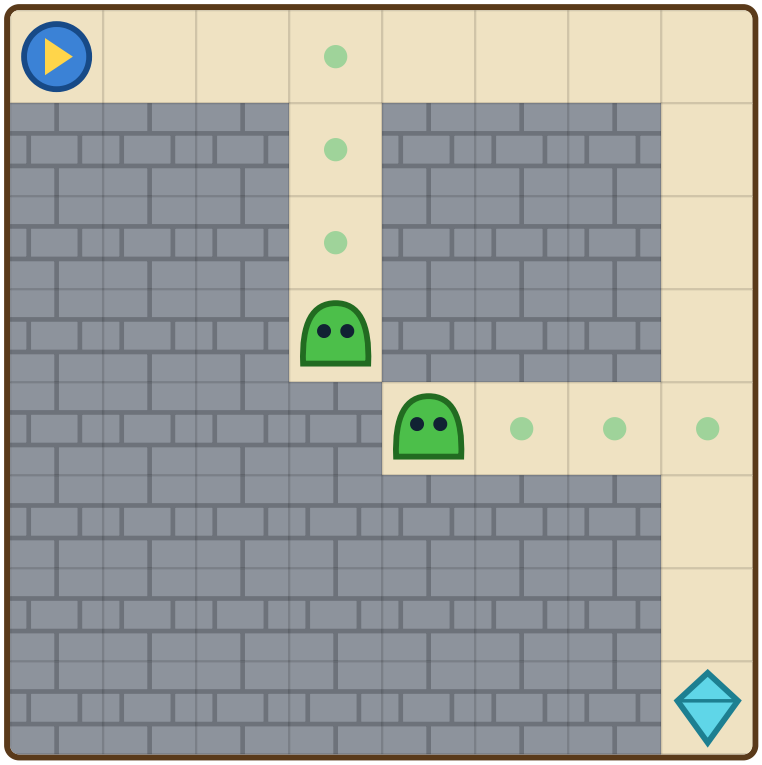
⚠ **Common mistake**

If the “if” is outside the loop, it checks only once.

🎬 Video: `videos/en/mazeenemy-02.mp4`

Stage 3: A bigger maze

Conditions | Nesting | Repeat until | Timing



Problem

The maze is bigger. If it is dangerous, wait. If not: move forward when the front is open, turn right when it is blocked.

🎯 Target: 6 blocks or fewer

🎯 **Goal**：An if inside an if

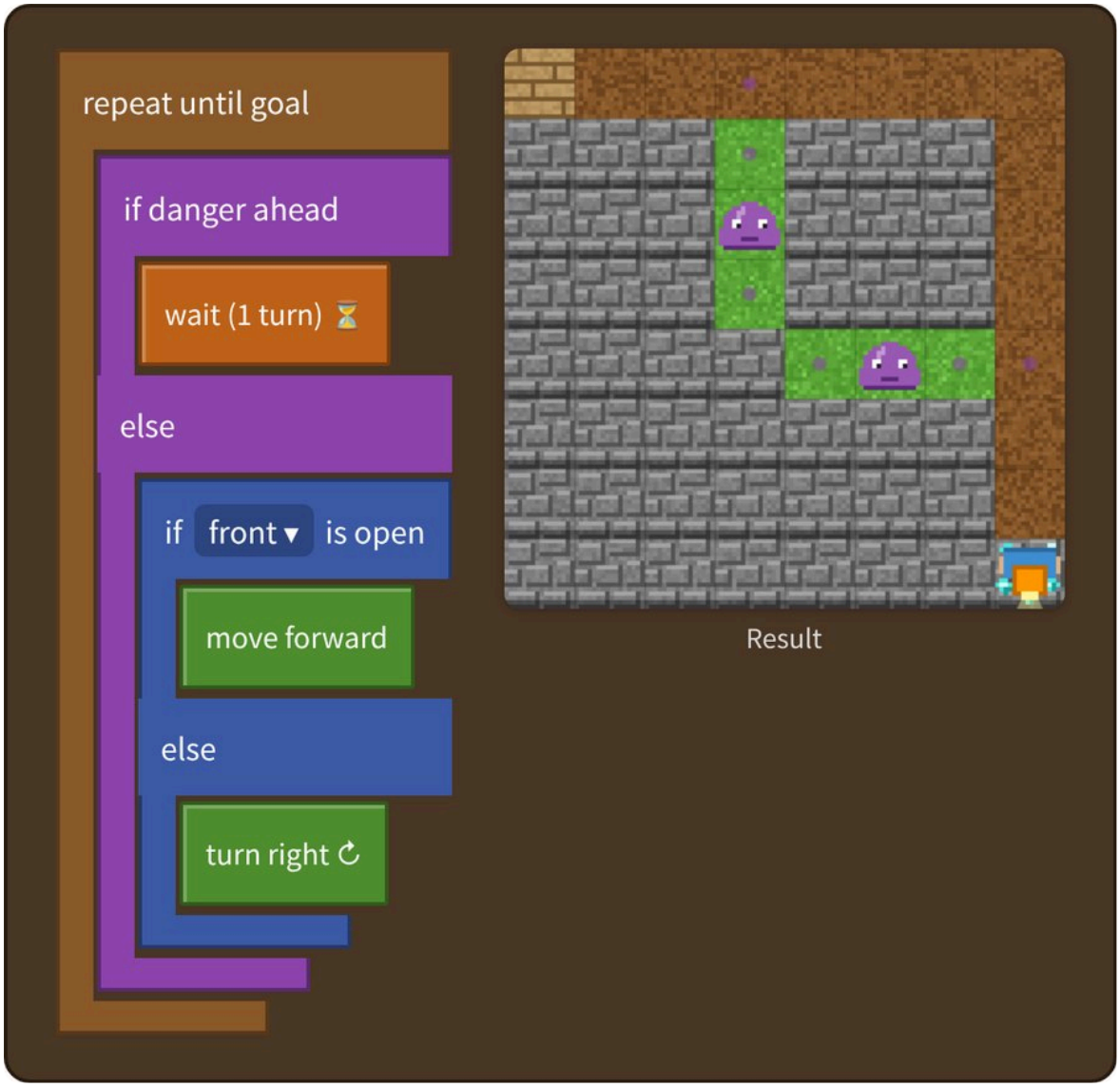
💡 **Hint (look if you get stuck)**

Repeat until goal: if danger ahead → wait, else → (if front is open → forward, else → turn right).

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: A bigger maze



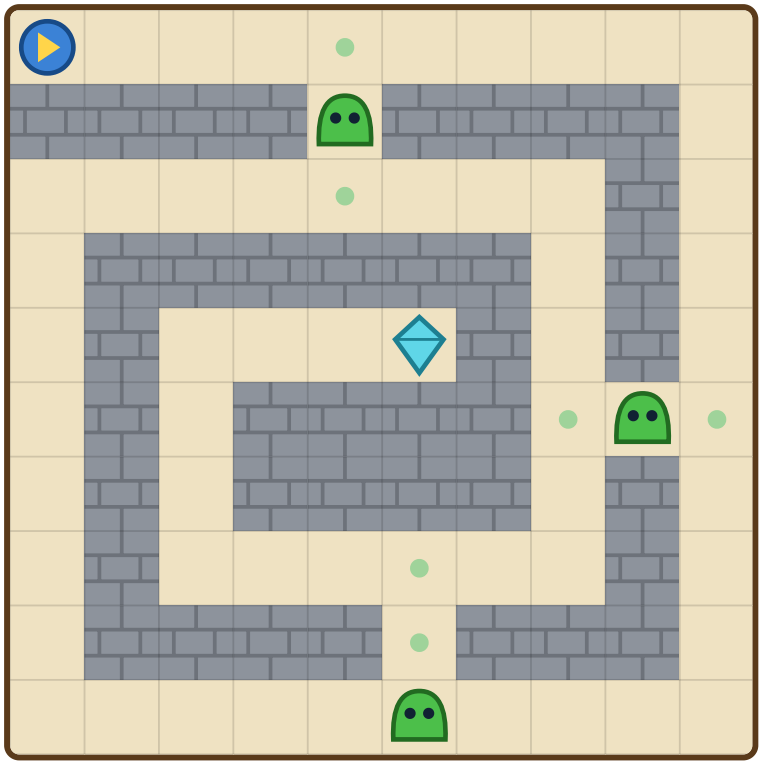
> ✨ **Key point**
>
> First check for danger. If it is safe, check if the front is open. The order matters.

> ⚠ **Common mistake**
>
> If you move without checking for danger, a slime catches you on the way.

🎬 Video: `videos/en/mazeenemy-03.mp4`

Stage 4: The spiral maze

Generalization | Repeat until | Timing



Problem

Last one: a 10×10 spiral with three slimes. Can the program from Stage 3 reach the goal?

🎯 Target: 6 blocks or fewer

🎯 **Goal**：The same program, a bigger maze

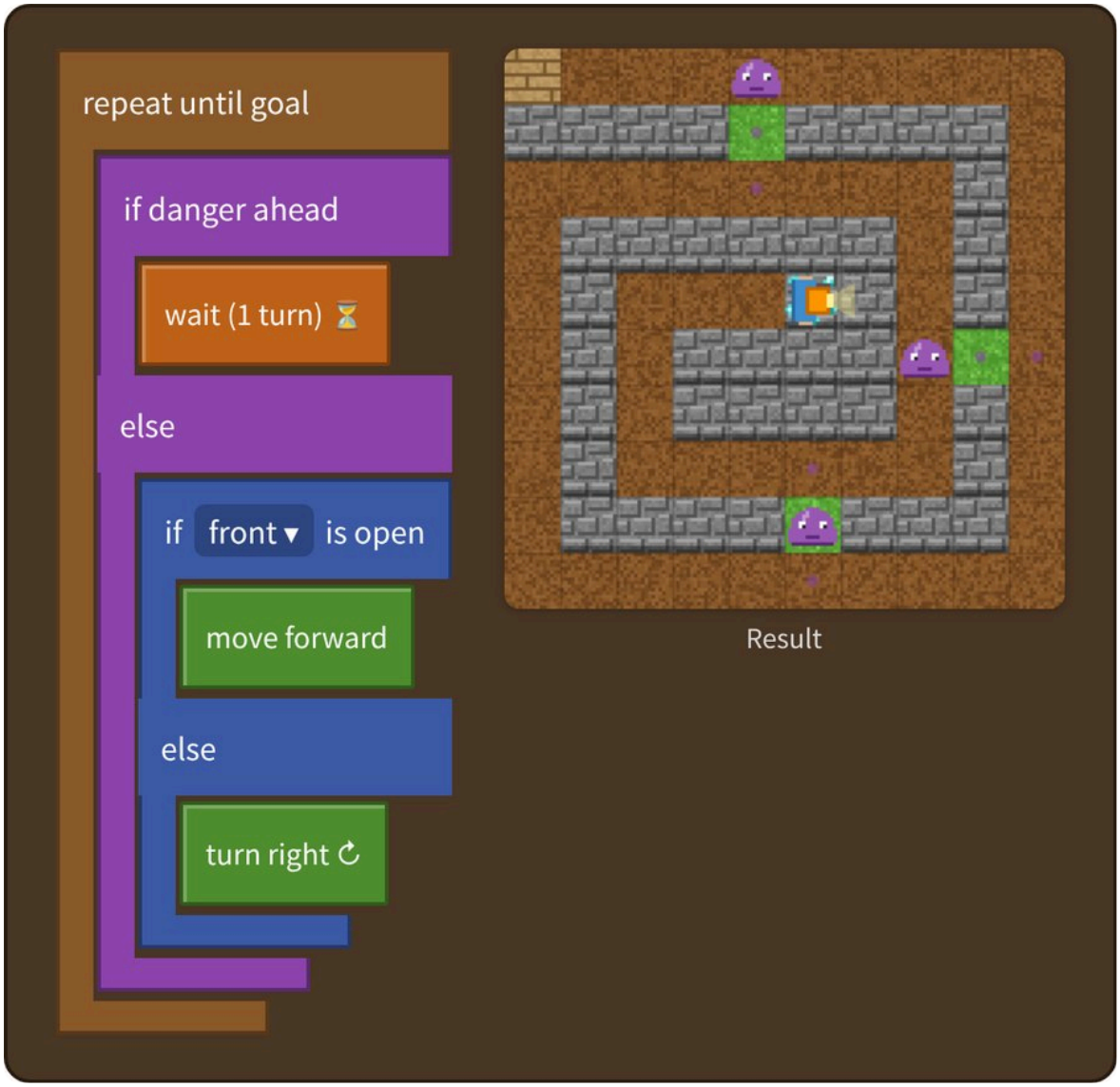
💡 **Hint (look if you get stuck)**

Use the same program as Stage 3. The slimes cross the path, so check for danger every time.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: The spiral maze



✨ Key point

The Stage 3 program works as it is. That is a method that works anywhere (generalization).

⚠ Common mistake

Without “wait”, one of the three slimes catches you.

🎬 Video: `videos/en/mazeenemy-04.mp4`

Column

Think about time, too

In the earlier mazes you only thought about “where to go”. With slimes, “**when** to go” matters too. Write the slime’s square for each turn in the worksheet table, and you will see the turn where you would bump into it.

Chapter 6

🐍 Mazes in Python

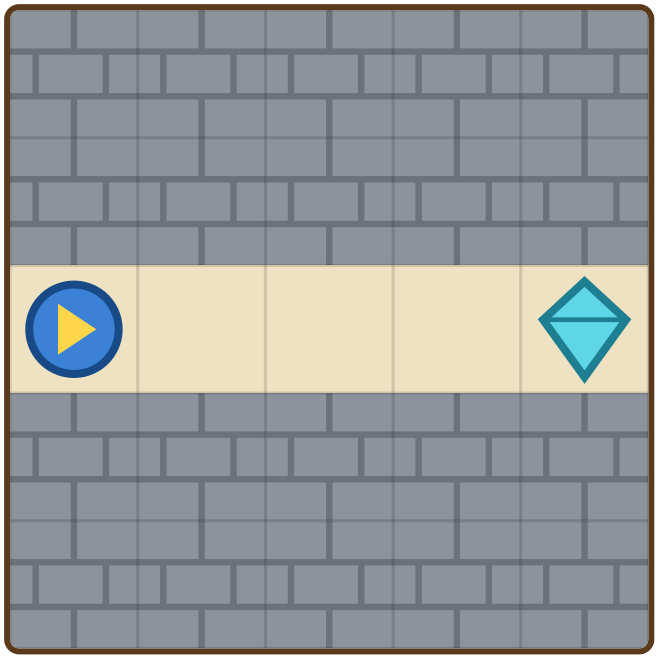
Programs you build with blocks can also be written in **Python**. “move forward” is `forward()`, “repeat 4 times” is `for i in range(4):`. Python is a real language used by programmers all over the world.

Commands in this chapter

forward() / turn_right() / turn_left() / for i in range(4): / while not at_goal(): / if is_open('front'): … else:

Stage 1: Go straight

Sequence | Python syntax (colon & indent)



Same maze as Block Maze stage 1

Problem

Type forward() to move the explorer to the diamond 💎. One line moves one square.

🎯 **Goal**：Write Python commands one line at a time

💡 **Hint (look if you get stuck)**

Write forward() 4 times, one per line.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go straight

```
forward()
forward()
forward()
forward()
```

✨ Key point

Python runs from the top line down. The “move forward” block = forward()

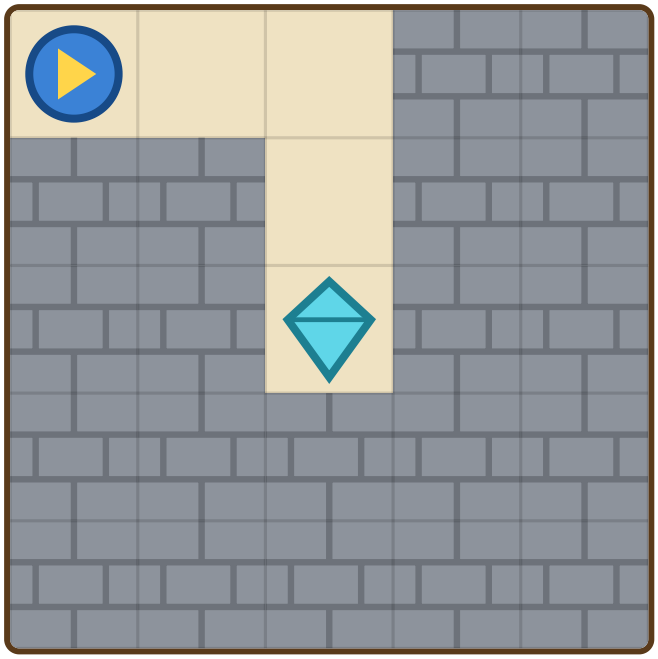
⚠ Common mistake

Writing forward without “()” causes an error.

🎬 Video: `videos/en/pymaze-01.mp4`

Stage 2: Let's turn

Sequence | Direction | Python syntax (colon & indent)



Same maze as Block Maze stage 2

Problem

turn_right() turns right, turn_left() turns left. Turning does not move you.

🎯 **Goal**：Use commands that change direction

💡 **Hint (look if you get stuck)**

forward() ×2 → turn_right() → forward() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Let's turn

```
forward()
forward()
turn_right()
forward()
forward()
```

✨ Key point

Turning does not move you. Don’t forget the “_” (underscore).

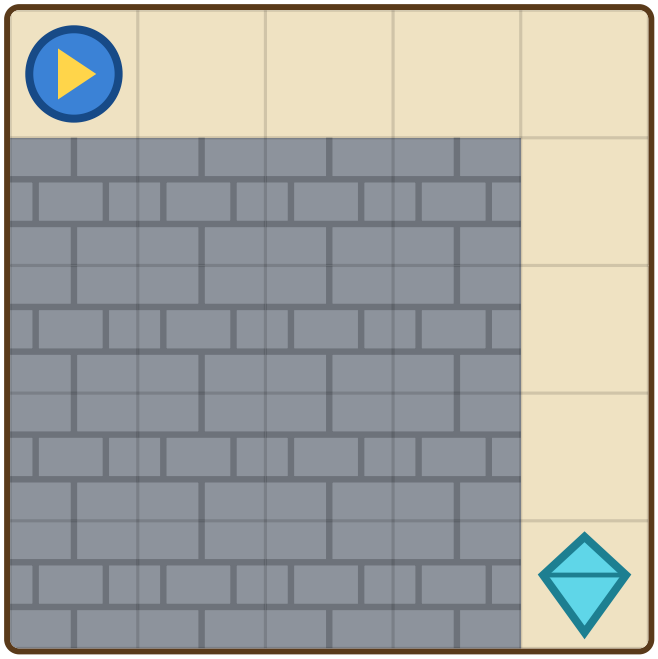
⚠ Common mistake

“turn right” (space) or “Turn_right” (capital) are different names.

🎬 Video: `videos/en/pymaze-02.mp4`

Stage 3: Repeat with for

Loops | Python syntax (colon & indent)



Same maze as Block Maze stage 4

Problem

“for i in range(4):” repeats the lines under it 4 times. Indent those lines with 4 spaces!

🎯 **Goal**：Write a loop with for and indentation

💡 **Hint (look if you get stuck)**

for i in range(4): → forward() (indented) → turn_right() → the same loop again

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Repeat with for

```
for i in range(4):
    forward()
turn_right()
for i in range(4):
    forward()
```

✨ Key point

Lines indented after “:” are inside the loop. Un-indent to leave the loop.

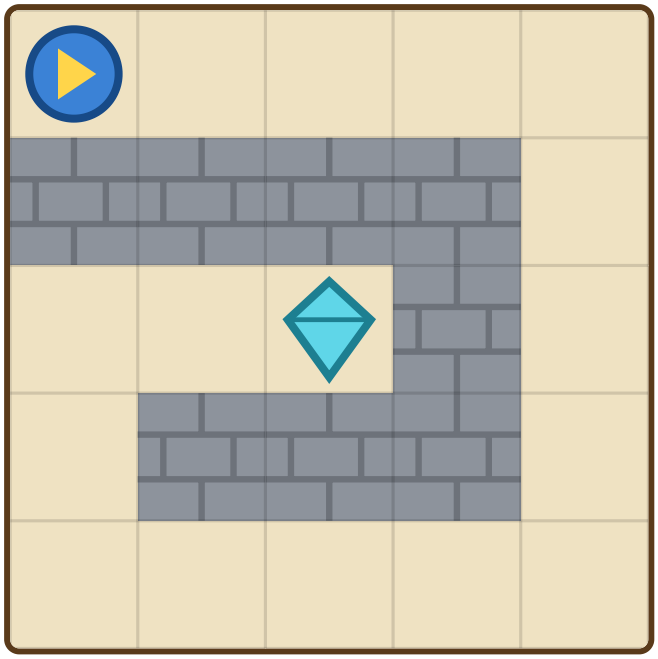
⚠ Common mistake

Forgetting the final “:”. Indenting turn_right() too makes it turn 4 times.

🎬 Video: `videos/en/pymaze-03.mp4`

Stage 4: while and if

Conditions | Repeat until | Python syntax (colon & indent)



Same maze as Block Maze stage 6

Problem

Repeat with “while not at_goal():” and decide with “if is_open('front'):”. else runs when the front is blocked.

🎯 **Goal**：Combine while with if / else

💡 **Hint (look if you get stuck)**

while not at_goal(): → if is_open('front'): forward() → else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: while and if

```
while not at_goal():
    if is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

not means “is not”. while not at_goal() = “repeat while you are not at the goal”.

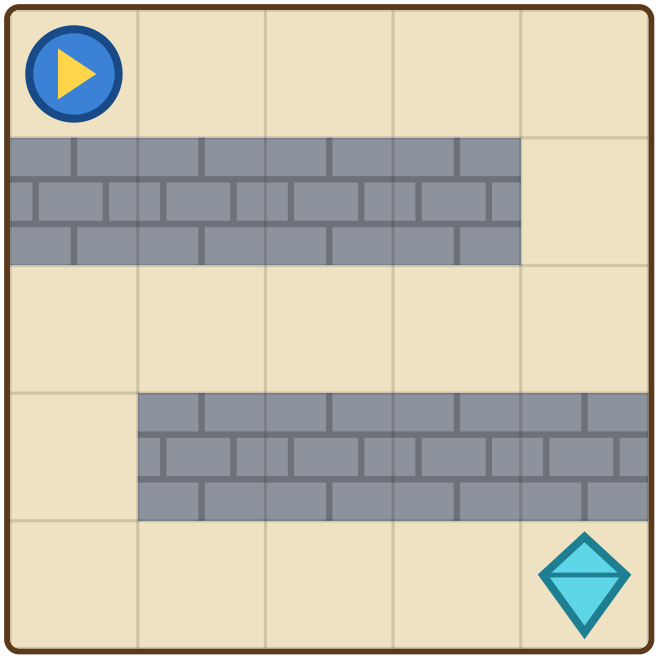
⚠ Common mistake

Misaligned if/else causes an error. Write else: at the same depth as if.

🎬 Video: `videos/en/pymaze-04.mp4`

Stage 5: Three choices with elif

Conditions | Repeat until | Nesting | Generalization | Python syntax (colon & indent)



Same maze as Block Maze stage 7

Problem

Python has elif (“else if”). Check the front, then the right, and otherwise turn left.

🎯 **Goal**：Split into three cases with elif

💡 **Hint (look if you get stuck)**

if is_open('front'): ... elif is_open('right'): ... else: ...

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Three choices with elif

```
while not at_goal():
    if is_open('front'):
        forward()
    elif is_open('right'):
        turn_right()
    else:
        turn_left()
```

✨ Key point

elif means “else if”. Putting an “if” block inside “else” becomes elif in Python.

⚠ Common mistake

Writing “else if” is an error in Python. Changing the order takes a different path.

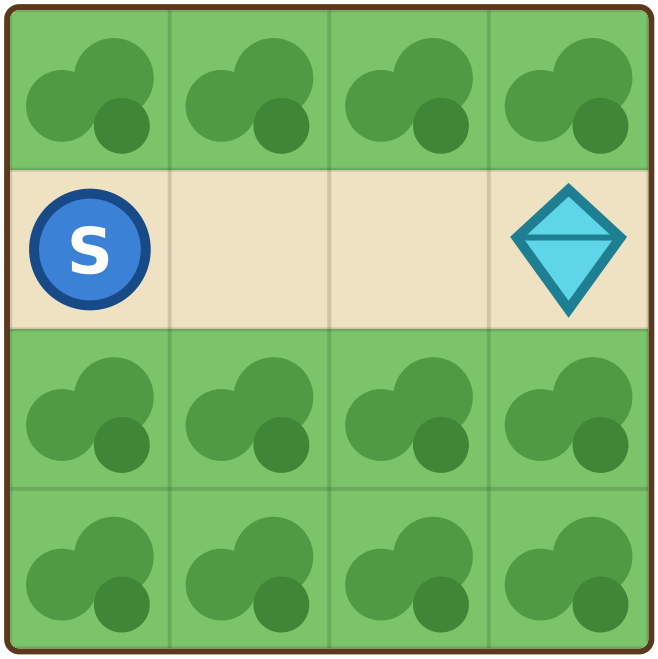
🎬 Video: `videos/en/pymaze-05.mp4`

🐍 Python Arrow Maze

The Arrow Maze in Python. Use the four commands up(), down(), left(), right() and for.

Stage 1: Go with right()

Sequence | Python syntax (colon & indent)



Same maze as Arrow Maze stage 1

Problem

right() moves one square to the right. Write it 3 times, one per line, to reach the diamond 💎.

🎯 **Goal**：Write right()

💡 **Hint (look if you get stuck)**

right() on 3 lines.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go with right()

```
right()
right()
right()
```

✨ Key point

Python has one command per line. Three lines of right() move 3 squares.

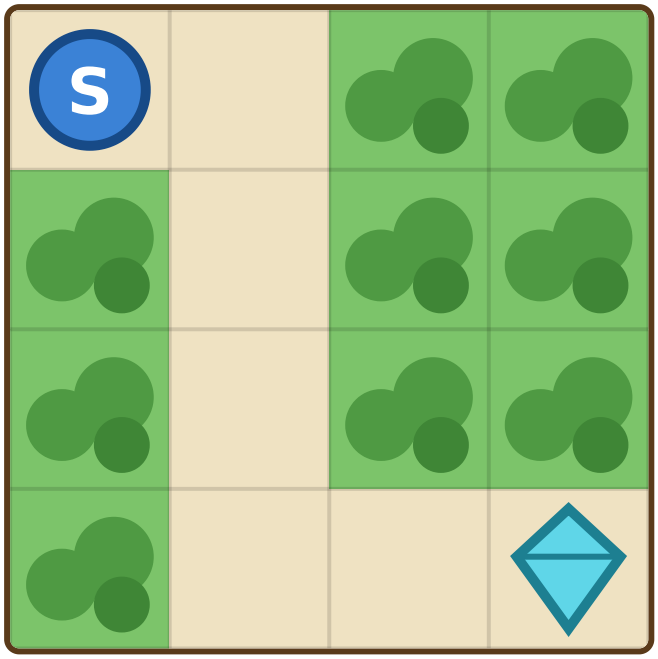
⚠ Common mistake

Writing just right without “()” causes a SyntaxError.

🎬 Video: `videos/en/pymazejr-01.mp4`

Stage 2: Go down with down()

Sequence | Direction | Python syntax (colon & indent)



Same maze as Arrow Maze stage 2

Problem

down() moves one square down.

🎯 **Goal**：Use down() too

💡 **Hint (look if you get stuck)**

right() → down() ×3 → right() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Go down with down()

```
right()
down()
down()
down()
right()
right()
```

✨ Key point

Write right() and down() in the order you go.

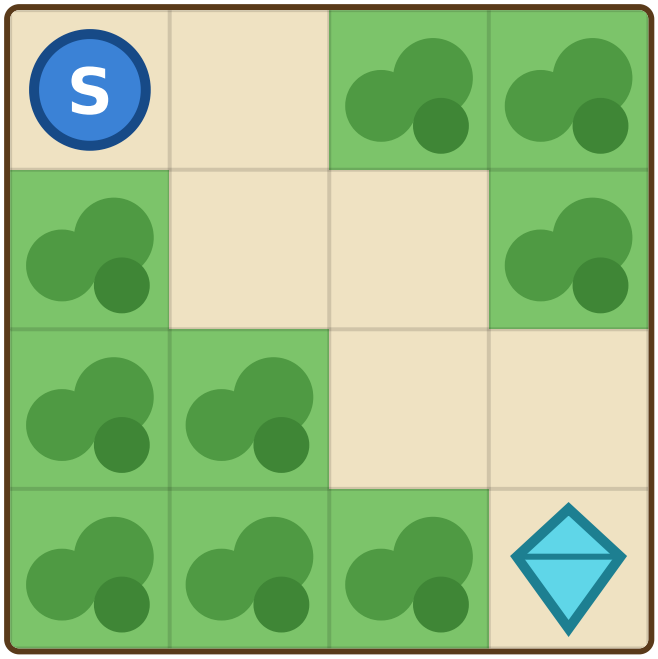
⚠ Common mistake

Down() with a capital letter causes a NameError. Python cares about capital letters.

🎬 Video: `videos/en/pymazejr-02.mp4`

Stage 3: Stairs

Sequence | Patterns | Python syntax (colon & indent)



Same maze as Arrow Maze stage 3

Problem

right() and down(), one after the other.

🎯 **Goal**：Write them alternately

💡 **Hint (look if you get stuck)**

right() down() right() down() right() down()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Stairs

```
right()
down()
right()
down()
right()
down()
```

✨ Key point

right() and down(), 3 times each, one after the other.

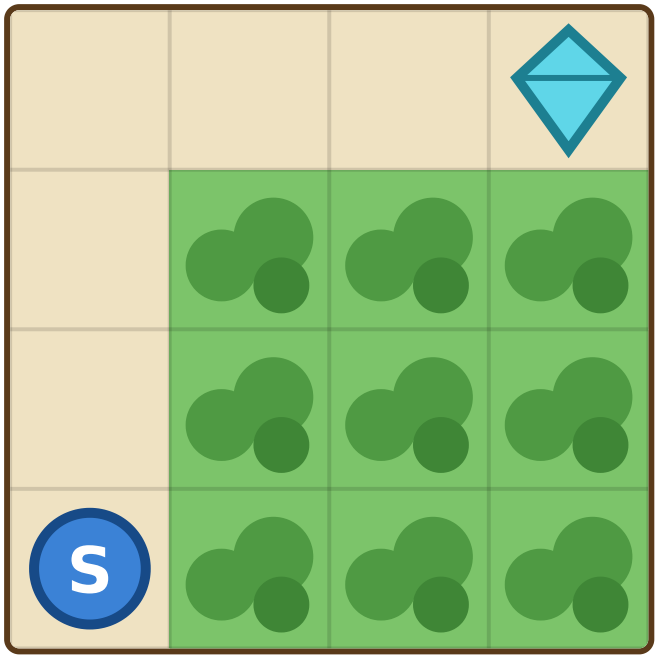
⚠ Common mistake

If the order is swapped, you bump into the bushes.

🎬 Video: `videos/en/pymazejr-03.mp4`

Stage 4: Go up with up()

Sequence | Direction | Python syntax (colon & indent)



Same maze as Arrow Maze stage 4

Problem

up() moves one square up.

🎯 **Goal**：Go up with up()

💡 **Hint (look if you get stuck)**

up() ×3 → right() ×3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Go up with up()

```
up()
up()
up()
right()
right()
right()
```

✨ **Key point**

Three lines of up(), then three lines of right().

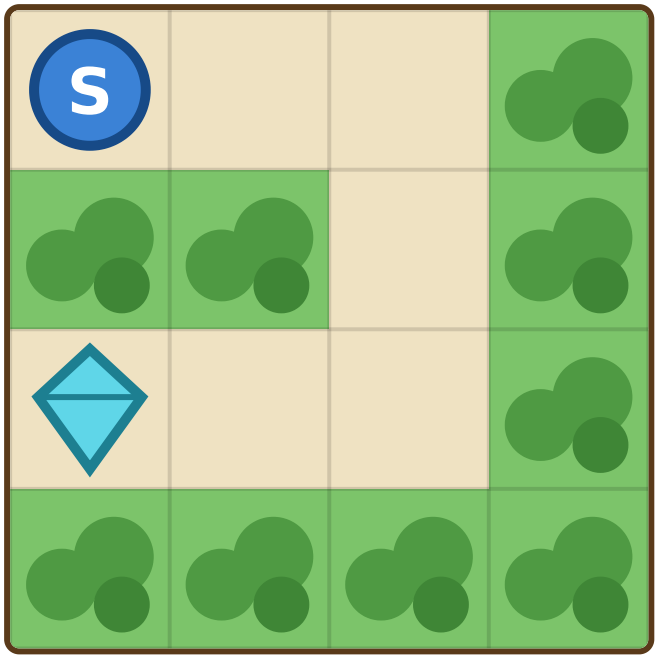
⚠ **Common mistake**

A space at the start of a line causes an IndentationError. Only lines inside for are indented.

🎬 Video: `videos/en/pymazejr-04.mp4`

Stage 5: Use left() too

Sequence | Direction | Python syntax (colon & indent)



Same maze as Arrow Maze stage 5

Problem

left() moves one square to the left. Use all four.

🎯 **Goal**：Use left() too

💡 **Hint (look if you get stuck)**

right() ×2 → down() ×2 → left() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Use left() too

```
right()
right()
down()
down()
left()
left()
```

✨ Key point

right() → down() → left(). Now you have used all four commands.

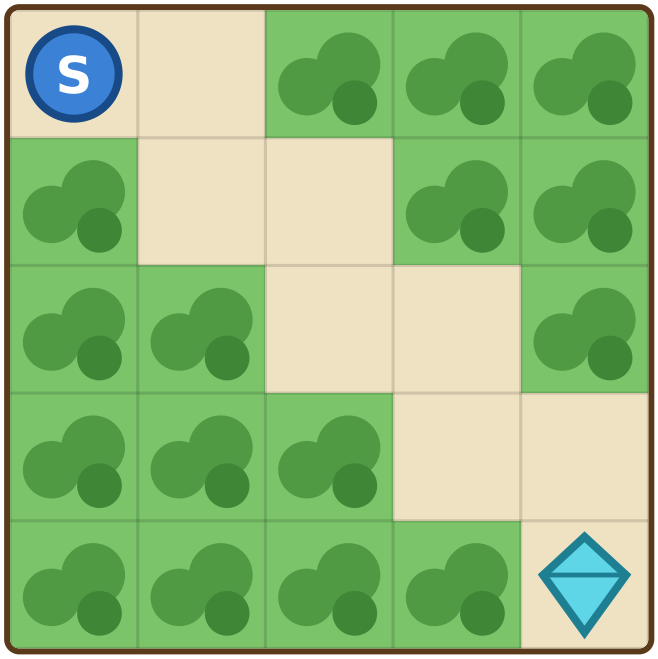
⚠ Common mistake

left and right are easy to mix up. Think of the left and right of the screen.

🎬 Video: `videos/en/pymazejr-05.mp4`

Stage 6: Repeat with for

Loops | Python syntax (colon & indent)



Same maze as Arrow Maze stage 6

Problem

“for i in range(4):” repeats the lines under it 4 times. Indent those lines with 4 spaces.

🎯 **Goal**：Repeat with for

💡 **Hint (look if you get stuck)**

for i in range(4): → (indented) right() and down()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Repeat with for

```
for i in range(4):
    right()
    down()
```

✨ Key point

Indent the 2 lines under for i in range(4): by 4 spaces. Both lines repeat.

⚠ Common mistake

If down() is not indented, only right() repeats.

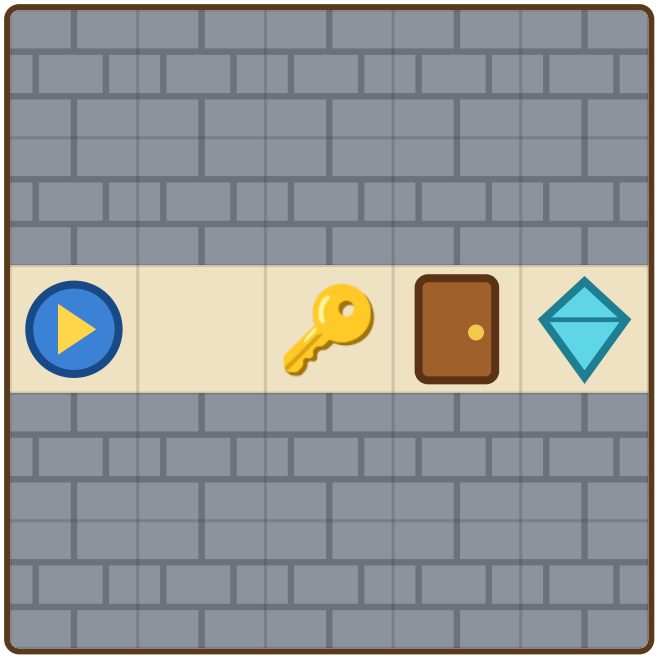
🎬 Video: `videos/en/pymazejr-06.mp4`

🐍 Python Keys & Doors

Keys & Doors in Python. Use pick_up(), open_door() and if with key_here() and door_ahead().

Stage 1: pick_up() and open_door()

Sequence | State | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 1

Problem

On the key 🔑, pick_up(). In front of the door 🚪, open_door(). Then you can go through.

🎯 **Goal**：pick_up() and open_door()

💡 **Hint (look if you get stuck)**

forward() ×2 → pick_up() → open_door() → forward() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: pick_up() and open_door()

```
forward()
forward()
pick_up()
open_door()
forward()
forward()
```

✨ Key point

pick_up() makes 1 key and open_door() makes 0. Your item count changes.

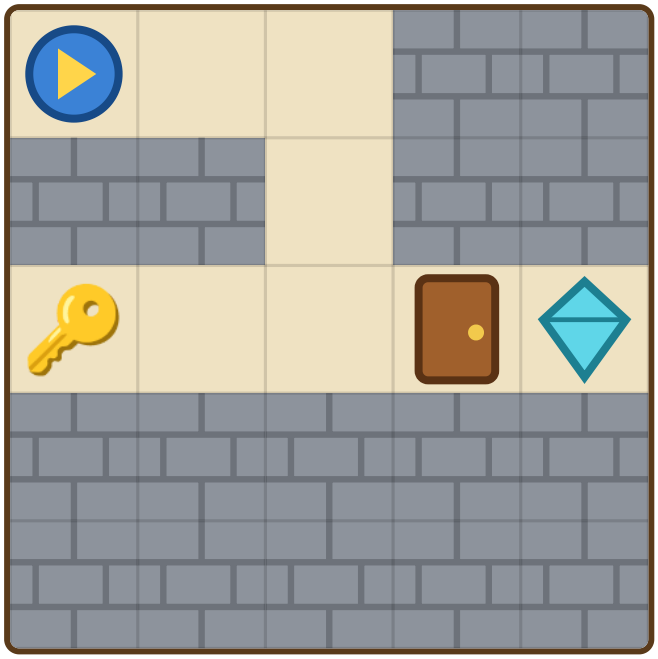
⚠ Common mistake

pickup() or open door() causes an error. Don’t forget the “_”.

🎬 Video: `videos/en/pymazekey-01.mp4`

Stage 2: Where is the key?

Sequence | State | Decomposition | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 2

Problem

Get the key first. Without a key, open_door() does not work.

🎯 **Goal**：Write the side trip in order

💡 **Hint (look if you get stuck)**

Walk to the key, pick_up(), turn_right() twice and go back.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 2: Where is the key?

```
forward()
forward()
turn_right()
forward()
forward()
turn_right()
forward()
forward()
pick_up()
turn_right()
turn_right()
forward()
forward()
open_door()
forward()
forward()
```

✨ Key point

Go to the key → pick_up() → turn_right() twice → to the door. Write a long plan one line at a time.

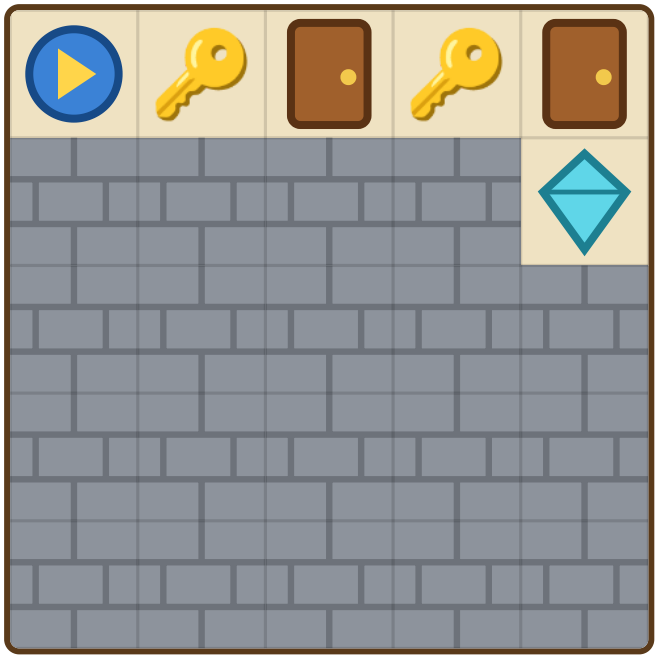
⚠ Common mistake

open_door() before pick_up() gives “You have no key.”

🎬 Video: `videos/en/pymazekey-02.mp4`

Stage 3: Two doors with for

Loops | Patterns | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 3

Problem

Repeat “forward() → pick_up() → open_door() → forward()” with for.

🎯 **Goal**：Four lines inside for

💡 **Hint (look if you get stuck)**

for i in range(2): (4 lines, indented) → turn_right() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 3: Two doors with for

```
for i in range(2):
    forward()
    pick_up()
    open_door()
    forward()
turn_right()
forward()
```

✨ **Key point**

All 4 lines inside for are indented by 4 spaces. The last 2 lines are not.

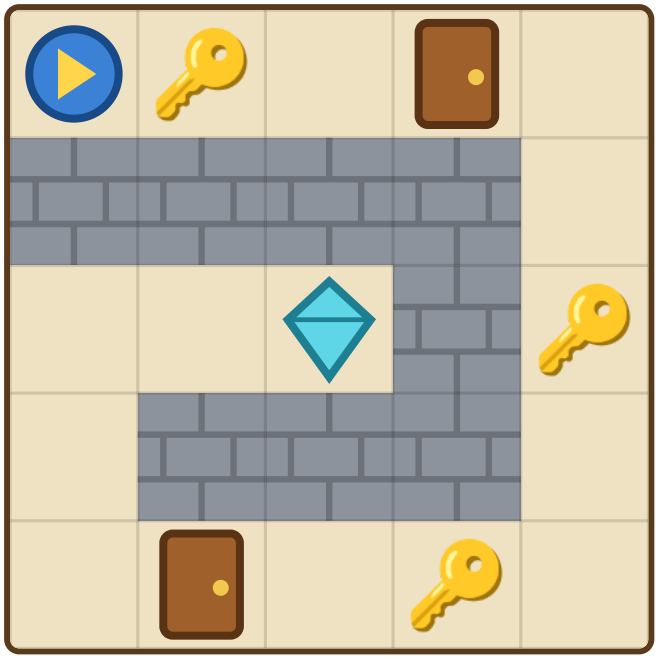
⚠ **Common mistake**

If turn_right() is indented too, you turn twice and go back.

🎬 Video: `videos/en/pymazekey-03.mp4`

Stage 4: key_here() and door_ahead()

Conditions | Repeat until | State | Generalization | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 4

Problem

key_here() checks for a key where you stand. door_ahead() checks for a door in front. Use them inside “while not at_goal():”.

🎯 **Goal**：Check with key_here() and door_ahead()

💡 **Hint (look if you get stuck)**

while not at_goal(): if key_here(): pick_up() / if door_ahead(): open_door() / if is_open('front'): forward() else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: key_here() and door_ahead()

```
while not at_goal():
    if key_here():
        pick_up()
    if door_ahead():
        open_door()
    if is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

Two ifs in a row, then if … else. Inside while, everything is checked again each time.

⚠ Common mistake

Forgetting “()” after key_here causes a SyntaxError. Checks also need ().

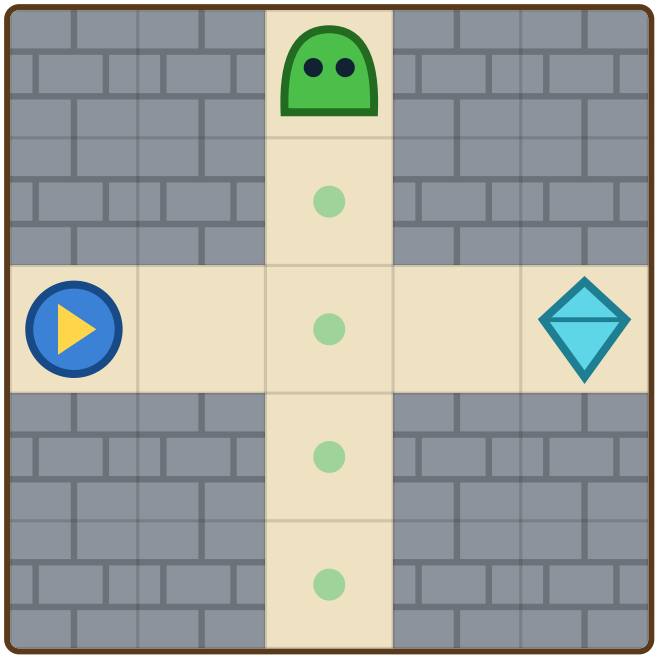
🎬 Video: `videos/en/pymazekey-04.mp4`

🐍 Python Slime Maze

Dodge the Slime Maze in Python. Use wait(), danger_ahead() and if, elif, else.

Stage 1: wait()

Sequence | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 1

Problem

wait() makes the explorer wait for 1 turn. The slime moves 1 square every turn. Time your move!

🎯 **Goal**：Wait with wait()

💡 **Hint (look if you get stuck)**

forward() → wait() ×2 → forward() ×3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: wait()

```
forward()
wait()
wait()
forward()
forward()
forward()
```

✨ Key point

wait() waits one turn. The slime moves one square meanwhile.

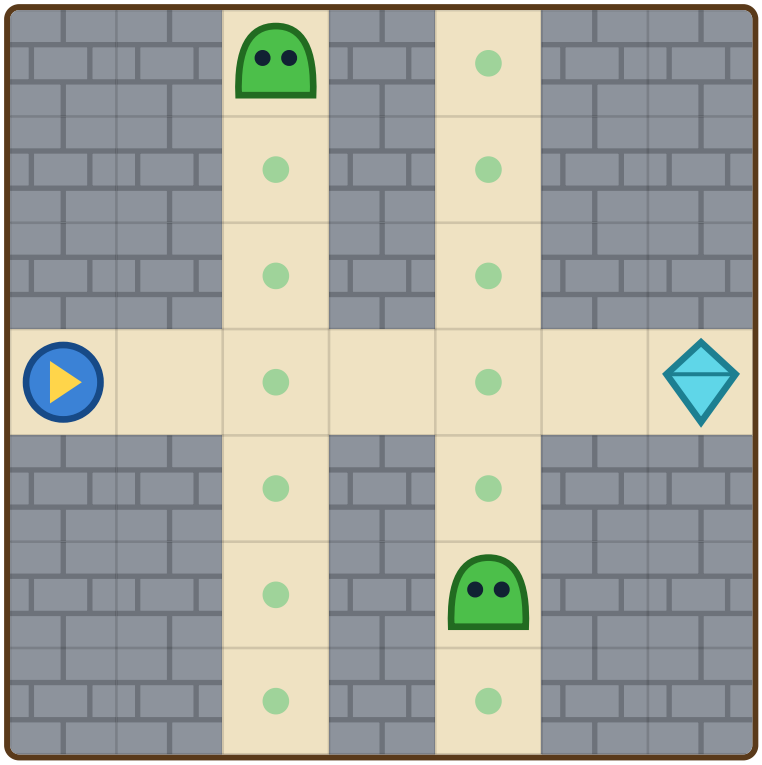
⚠ Common mistake

wait(2) causes a TypeError. To wait twice, write wait() on 2 lines.

🎬 Video: `videos/en/pymazeenemy-01.mp4`

Stage 2: if danger_ahead():

Conditions | Repeat until | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 2

Problem

danger_ahead() is True when a slime is in front of you or will come there next. If so, wait(); else, forward().

🎯 **Goal**：Check with if danger_ahead():

💡 **Hint (look if you get stuck)**

while not at_goal(): if danger_ahead(): wait() else: forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: if danger_ahead():

```
while not at_goal():
    if danger_ahead():
        wait()
    else:
        forward()
```

✨ **Key point**

danger_ahead() is True when it is dangerous. if ... else decides between waiting and moving.

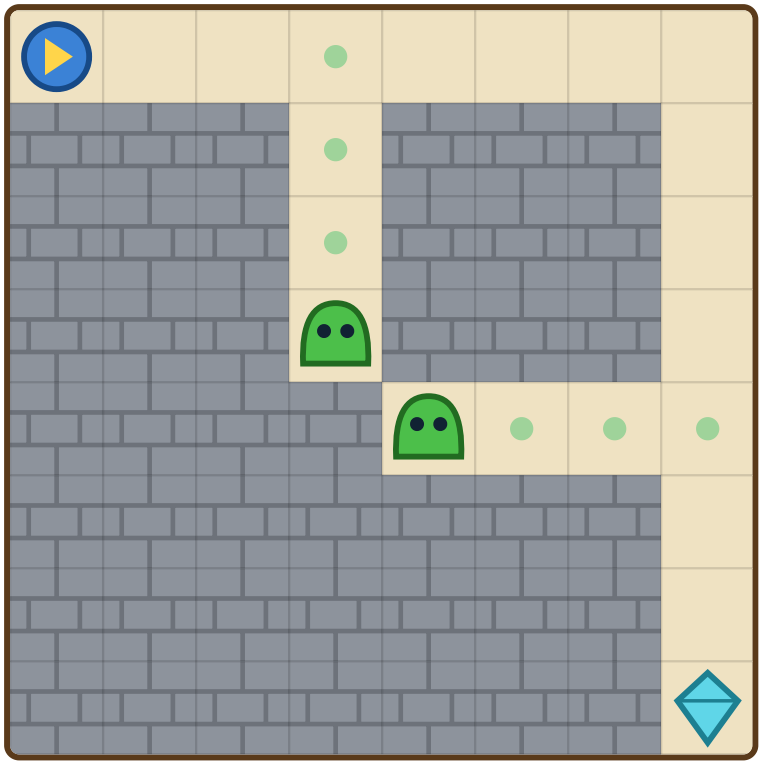
⚠ **Common mistake**

Forgetting the “:” after else causes a SyntaxError.

🎬 Video: `videos/en/pymazeenemy-02.mp4`

Stage 3: A bigger maze with elif

Conditions | Nesting | Repeat until | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 3

Problem

If danger, wait(). elif the front is open, forward(). else turn_right().

🎯 **Goal**：Split into three with elif

💡 **Hint (look if you get stuck)**

while not at_goal(): if danger_ahead(): wait() / elif is_open('front'): forward() / else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: A bigger maze with elif

```
while not at_goal():
    if danger_ahead():
        wait()
    elif is_open('front'):
        forward()
    else:
        turn_right()
```

✨ **Key point**

if (danger) → elif (front is open) → else (turn right). They are checked from the top.

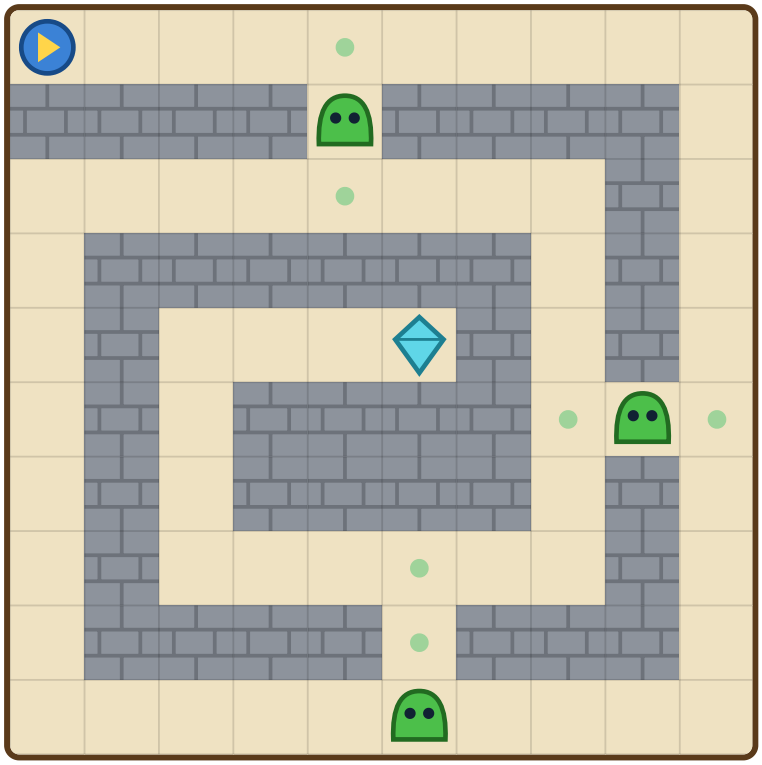
⚠ **Common mistake**

If you change elif to if, you may move right after waiting and get caught.

🎬 Video: `videos/en/pymazeenemy-03.mp4`

Stage 4: The spiral maze

Generalization | Repeat until | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 4

Problem

A 10×10 spiral with three slimes. Does your program from Stage 3 work here too?

🎯 **Goal**：The same code, a bigger maze

💡 **Hint (look if you get stuck)**

The Stage 3 program works as it is.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: The spiral maze

```
while not at_goal():
    if danger_ahead():
        wait()
    elif is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

Use the Stage 3 code as it is. It works in any maze.

⚠ Common mistake

Forgetting the quotes in is_open('front') causes an error.

🎬 Video: `videos/en/pymazeenemy-04.mp4`

Column

Three Python rules

① Every command ends with `()`. ② Lines starting with `for`, `while` or `if` end with `:` (a colon). ③ The lines inside are indented by **4 spaces**. If you get an error, check these three first. Capital and small letters are different, too.

Well done!

You worked hard to the end! Let’s look back at the ideas you used in this book.

Idea	Where did it appear?
Order	Commands run from top to bottom (every maze)
Loops	Pack the same moves together (Block Maze stages 4–5)
If (conditions)	Look ahead and decide (Block Maze stages 6–8, keys, slimes)
State	How many keys you carry (Keys & Doors)
Timing	Dodge with “wait” (Slime maze)
Algorithm	Steps that solve any maze (Block Maze stage 8)
Python	Write the same thing in words (code)

When you clear every stage, get your certificate from “⚙ Settings” → “📜 Certificate” in the app. Next, try “Block Drawing”, “Space Star Hunt” and “Break & Build Quest”!

PART 2

Part 2 Guide for teachers and parents

About this guide

Part 1 is written in simple language so that students can read it on their own. Part 2 is for teachers who plan and run lessons, and for parents who support learning at home.

This book collects the eight maze apps (4 block, 4 Python; 41 stages in total) of a free, browser-based block programming curriculum published on GitHub Pages. It is designed for elementary programming education focused on logical thinking (“computational thinking”), and as an introduction to programming in junior high school technology classes.

Aims

- Think of the steps to reach a goal as a combination of sequence, loops and conditions.
- Find and fix the cause of a problem from the result of running the program (trial and error, debugging).
- Express the same task in both blocks and Python (a text language) and notice how they correspond.

Overview of the stages

App	#	Stage	Ideas
🧭 Arrow Maze	1	Go right	Sequence
	2	Go down	Sequence, Direction
	3	Stairs	Sequence, Patterns
	4	Go up	Sequence, Direction
	5	Go around	Sequence, Direction
	6	Use repeat	Loops, Patterns
⛏️ Block Maze	1	Go straight	Sequence
	2	Let's turn	Sequence, Direction
	3	A long path	Sequence, Direction
	4	Use a loop	Loops
	5	Climb the stairs	Loops, Patterns
	6	If...	Conditions, Repeat until
	7	Right? Left?	Conditions, Nesting
	8	Any maze!	Generalization, Conditions, Repeat until
🗝️ Keys & Doors Maze	1	Pick up the key, open the door	Sequence, State
	2	Where is the key?	Sequence, State, Decomposition
	3	Two doors	Loops, Patterns, State
	4	A thinking explorer	Conditions, Repeat until, State, Generalization
👾 Dodge the Slime Maze	1	Wait and dodge	Sequence, Timing
	2	Wait if it is dangerous	Conditions, Repeat until, Timing
	3	A bigger maze	Conditions, Nesting, Repeat until, Timing
	4	The spiral maze	Generalization, Repeat until, Timing
🐍 Python Maze	1	Go straight	Sequence, Python syntax (colon & indent)
	2	Let's turn	Sequence, Direction, Python syntax (colon & indent)
	3	Repeat with for	Loops, Python syntax (colon & indent)
	4	while and if	Conditions, Repeat until, Python syntax (colon & indent)
	5	Three choices with elif	Conditions, Repeat until, Nesting, Generalization, Python syntax (colon & indent)
🐍 Python Arrow Maze	1	Go with right()	Sequence, Python syntax (colon & indent)
	2	Go down with down()	Sequence, Direction, Python syntax (colon & indent)
	3	Stairs	Sequence, Patterns, Python syntax (colon & indent)
	4	Go up with up()	Sequence, Direction, Python syntax (colon & indent)
	5	Use left() too	Sequence, Direction, Python syntax (colon & indent)
	6	Repeat with for	Loops, Python syntax (colon & indent)
🐍 Python Keys & Doors	1	pick_up() and open_door()	Sequence, State, Python syntax (colon & indent)
	2	Where is the key?	Sequence, State, Decomposition, Python syntax (colon & indent)
	3	Two doors with for	Loops, Patterns, Python syntax (colon & indent)

	4	key_here() and door_ahead()	Conditions, Repeat until, State, Generalization, Python syntax (colon & indent)
🐍 Python Slime Maze	1	wait()	Sequence, Timing, Python syntax (colon & indent)
	2	if danger_ahead():	Conditions, Repeat until, Timing, Python syntax (colon & indent)
	3	A bigger maze with elif	Conditions, Nesting, Repeat until, Timing, Python syntax (colon & indent)
	4	The spiral maze	Generalization, Repeat until, Timing, Python syntax (colon & indent)

Suggested order and lesson count

Combine the apps according to grade and experience. Each stage takes from a few minutes to about 20 minutes.

Students	Apps	Lessons	Key ideas
Grades 1–2	Arrow Maze 1–6	1–2	Order, counting
Grades 2–3	Block Maze 1–5	2	Order, direction, loops
Grades 4–6	Block Maze 6–8, Keys & Doors, Slime	3–4	Conditions, state, timing, generalization
Grade 5 – junior high	Python mazes (4 apps)	3–5	Blocks ↔ code, syntax, reading errors

Students progress at different speeds. For fast finishers: try the block limit, act as a “hint giver” for friends, or solve the same maze in the Python version.

Example flow of a 45-minute lesson

Time	Activity	Teacher support
Intro 5 min	Review the last lesson; share today’s goal (Worksheet 1).	You can write the book’s “Goal” on the board as it is.
Plan 5 min	Look at the maze and predict the steps (Worksheets 3–7).	Have students trace the path with a finger before touching the device.
Try 20 min	Try in the app; find and fix the cause if it fails.	Don’t give the answer; ask “Where did it stop?” Allow pair talk.
Share 10 min	Show different solutions; think about shorter programs.	Compare 2–3 programs on the big screen.
Reflect 5 min	Write a reflection (Worksheet 1).	Separate “what I learned” and “what I want to try next”.

Teaching points for each app

🧭 Arrow Maze (lower grades)

- **Aim:** focus only on “order” by lining up arrows, without thinking about direction.
- **Common difficulty:** miscounting squares (counting the start square). → “Don’t count the square the explorer is on.” Have students touch and count one square at a time.
- **Question to ask:** “If you have one arrow too few, where will it stop?”

⛏️ Block Maze

- **Aim:** move step by step from sequence → loops → conditions → generalization (algorithm).
- **Common difficulties:** ① thinking “turn right” also moves forward (it only turns); ② confusing left and right (screen right vs. the explorer’s right); ③ in stage 8, making steps that work only for one maze.
- **Support:** for ②, rotate the book or tablet, or have students turn their bodies. For ③, have students say a rule that works for any maze before building it (the right-hand rule).

🗝️ Keys & Doors

- **Aim:** think while keeping track of state (“how many am I carrying now?”) — a basis for variables.
- **Common difficulties:** picking up when not on a key, walking to a door without a key, losing count of keys.
- **Support:** have students write the number of keys after each move in the Worksheet 6 table, and watch the item display on screen.

👾 Dodge the Slime

- **Aim:** think about “when” as well as “where”, then generalize with “if danger ahead”.
- **Common difficulties:** too few or too many “wait”s; missing that slimes also move when you only turn.
- **Support:** show the run one turn at a time and have students record the slime’s position in a table.

🐍 Python mazes

- **Aim:** notice how blocks and code correspond, and that the same thinking works in text.

- **Common difficulties:** missing `()` or `:` (SyntaxError), wrong indentation (IndentationError), capital letters or spelling (NameError).
- **Support:** present errors as hints, not failures, and have students log them on Worksheet 7. Building in block mode and comparing with the Python shown below also helps.

Assessment (example)

Area	Meets expectations (B)	Exceeds (A), e.g.
Knowledge & skills	Arranges blocks (code) correctly and clears the maze.	Uses loops and conditions to clear it with a short program.
Thinking, judgment & expression	Finds where the error is from the result and fixes it.	Explains a method that works for any maze and turns it into a program.
Attitude toward learning	Keeps trying even when it doesn’t work.	Shares ideas with friends and looks for better ways on their own.

Use the worksheets (1 reflection; 3–7 steps and records) and the app progress (8 check card, certificate) as evidence.

Using the apps and related materials

Requirements and tips

- Works in the browser on computers, tablets and smartphones. No installation needed (it loads the block library from the internet, so use it online).
- On narrow screens it switches to a phone layout automatically.
- Cleared stages are saved in that device’s browser. If devices are shared, check progress at the start of each lesson.
- **Teacher mode:** add `?sensei` to the end of the address to show an “Answer” button that loads the answer blocks.
- From “⚙ Settings” you can switch Japanese/English, use night mode, choose a maze (“Choose a maze…”) and show the certificate.

Related materials

Material	Contents
Guide (guide.html)	Goals, common mistakes, hints and answers for every task. This book is built from the same data.
Worksheets (PDF)	1 goal & reflection, 3 maze, 4 arrows, 6 keys & slime, 7 Python, 8 check card, and more.
Videos (videos.html)	Short videos showing how to solve each stage.
Lesson slides	02 Block Maze, 06 Python Maze, 09–14 maze variations (PowerPoint).
Answer list (ANSWERS.md)	Answer blocks and code for every stage.